

DEON: Runtime Guards for Policy-Safe Agentic Service Composition under Attack

Xuan-Bach Le¹ ^{✉*}, Phat T. Tran-Truong¹ , Duy Nguyen² , and Ngoc Khanh Tran Nguyen³ 

¹ Ho Chi Minh City University of Technology (HCMUT), VNU-HCM, Vietnam
`{lexuanbach, phat}@hcmut.edu.vn`

² University of Economics Ho Chi Minh City, Vietnam
`duynguyen.726202321440@st.ueh.edu.vn`

³ University of Science, VNU-HCM, Vietnam
`22120378@student.hcmus.edu.vn`

Abstract. Large language models increasingly orchestrate services, but attacker-controlled inputs can induce access-control violations, data leakage, omitted mandatory steps, and quota overruns. Prompt-based defences reduce these failures without eliminating them. We place two runtime checks between an untrusted proposer and service execution. Our *deontic policy automaton* checks each proposed action against explicit permissions, prohibitions, and obligations. Under correct catalogue labels, complete mediation, and an adequate policy, no action violating the policy’s safety fragment executes. Terminally pending obligations are detected rather than forced. A mislabelled action falls outside this guarantee. Our conformal guard handles residual violations, and we calibrate its detector threshold on labelled data. The pass-through bound holds only while deployment violation scores remain exchangeable with the calibration scores. Adaptive low-scoring rewrites invalidate the bound and are admitted. On DEON-BENCH, the automaton agrees with ground truth on **100%** of 900 traces across six violation classes. For content egress that the policy cannot identify, the calibrated layer admits **8.9%** of leaks against a **10%** target, rising to **32.8%** under adaptive obfuscation. Across six proposer models, our guard reduces attack success for policy-expressible injections from **87–100%** to **0%**, with microsecond-scale monitoring overhead. We release DEON-BENCH and the guard.

Keywords: Agentic service composition · Runtime verification · Conformal prediction · Prompt injection · Deontic logic.

* X.-B. Le and P. T. Tran-Truong contributed equally to this work.

✉ Corresponding author: Xuan-Bach Le, `lexuanbach@hcmut.edu.vn`.

Extended version of the ICSOC 2026 paper. Artifact: <https://github.com/lexuanbach/deon>.

1 Introduction

Service-oriented computing increasingly uses large language models to interpret goals, browse API catalogues, and select service calls step by step. This practice builds on tool-augmented models [28], agentic loops [36,31], and service ecosystems that encode access rules as executable policy [30,21]. Unlike a batch predictor, however, an agent causes side effects. Its retrieved documents and tool outputs may contain indirect prompt injections [12,15] that alter the actions it proposes. Those actions can violate access-control rules, expose private fields, omit required steps, or exceed quotas. Policy violation is a prominent LLM risk [23]. In our experiments, six proposers select a policy-violating action on **87–100%** of attacked tasks (§9).

Prompt-based defences such as self-reflection [31] reduce these failures without eliminating them. Static verification [6,19] can certify a fixed plan, but an agent’s observations and subsequent proposals change during execution. Selective prediction and conformal risk control [11,5,2,3] provide calibrated abstention, but do not represent service policies or sequential policy state. This gap calls for an execution-time guard that combines explicit policy enforcement with calibrated handling of residual risk.

We design DEON to apply two checks before each proposed action (§4, Fig. 2). A *deontic policy automaton* (DPA) evaluates permissions, prohibitions, and obligations from the executed prefix and the policy. It does not use the proposer’s rationale. A *conformal execution guard* handles violations that the policy cannot express by calibrating a threshold that targets $\mathbb{P}[\text{admit} \mid \text{violation}] \leq \alpha$. Actions above this threshold receive an ABSTAIN verdict. The deterministic guarantee is uniform over proposers and adversaries, but only for policy-expressible actions covered by the deployment assumptions. The statistical guarantee applies to the residual class only while deployment violation scores remain exchangeable with the calibration scores. An adaptive adversary can invalidate it by producing violations that score below the threshold.

Four conditions determine how these guarantees should be read (§5): correct and complete action labels (A1), complete mediation (A2), an adequate policy (A3), and exchangeable calibration data (A4). OpenAPI [22] and MCP [18] schemas do not currently encode all of the privacy-lattice, sink-clearance, and quota metadata required by A1. A deployment must therefore obtain these labels from a trusted catalogue and route every effectful action through the guard. Under-labelled or unmediated actions fall outside the deterministic result. The probability-zero statements and measured 0% results are consequently conditional on A1–A3, and the conformal results also require A4.

Classical mechanisms provide important parts of this design. XACML [20], usage control [24], and OPA/Rego [21] enforce declarative policies, but agentic orchestration also requires trace state and distrust of model-supplied metadata. For example, whether **charge** is admissible can depend on earlier authentication, quota use, and a pending receipt obligation. Content-dependent actions introduce a further residual: “export the report” may be allowed even though exporting a particular report is unsafe. DEON combines a stateful policy monitor

for the first class with calibrated abstention for the second. §11 compares this design with the established enforcement models.

We organize the evaluation around six questions. **RQ1** measures monitor correctness and conformal calibration. **RQ2** evaluates attack success across proposers and attributes the reduction to the deterministic and statistical layers. **RQ3** studies policy-inexpressible content egress under exchangeable and adaptive inputs. **RQ4** evaluates detector ensembles and Mondrian calibration. **RQ5** examines sensitivity to sample size, label noise, ties, detector quality, and drift, together with abstention recovery. **RQ6** examines correlated steps and scorer evolution. The ICSOC 2026 paper reports RQ1–RQ3. This version reports all six.

We make three contributions. First, we define a two-layer runtime guard over a typed service graph and specify its integration boundary (§3–§6). Second, we state the deployment assumptions explicitly and prove DPA soundness, marginal conformal admission, and an exact training-conditional bound in the probably approximately correct (PAC) sense, together with the corresponding attack-success bounds (§7, App. A). Third, we evaluate these claims on DEON-BENCH, including ground-truth traces, a specified feature-based content-egress detector, calibration stress tests, hardening methods, abstention recovery, and a six-model proposer study (§8, §9). The released artifact regenerates the deterministic experiments and checks the committed AgentDojo results. The latter use a non-deterministic local proposer and are not claimed to reproduce bit-for-bit.

2 Background and Preliminaries

A *service catalogue* is a finite set $\mathcal{A}$ of operations with machine-readable OpenAPI interfaces, and an *agent* proposes one action at a time to execute against the current state.

Deontic policy. Access and usage rules are naturally *deontic*: permitted, forbidden, or obligatory [35]. We use three modalities: *permission* P (conditional on established facts), *prohibition* F (subsuming role/attribute access control [30], lattice information flow [8], and quotas), and *obligation* O (one action forces another later, e.g. `charge` obliges `emitReceipt`), all expressible in Rego/OPA [21]. Permission, prohibition, and preconditions are monitor-*enforceable*, whereas obligations are a *liveness* property a blocking monitor can only *detect* (§7).

Runtime verification. A *runtime monitor* reports incrementally whether an executing trace satisfies a property [16,4], inspecting the *single* finite trace produced, the setting of an agent whose next action is chosen online by an untrusted proposer.

Information flow and quotas. Fields carry *privacy tags* from a lattice $(\mathcal{L}, \sqsubseteq)$ (e.g. `public` $\sqsubseteq$ `internal` $\sqsubseteq$ `pii` $\sqsubseteq$ `secret`). Explicit-flow safety forbids a field

labelled ℓ reaching a sink cleared for $\ell' \sqsubset \ell$ [8], and metered resources carry per-trace quotas, both prohibitions the DPA enforces.

Conformal prediction and selective risk. Conformal prediction turns any score into a distribution-free, finite-sample guarantee under *exchangeability* [34,1]: an empirical-quantile threshold on a calibration sample controls a target error rate, and *Learn-then-Test* and *conformal risk control* extend it to a user-specified risk [2,3]. With a *reject/abstain* option [5,11] the rule acts only when confident. Marginal coverage holds *in expectation*, and *training-conditional* (PAC) coverage holds with probability $1 - \delta$ over an i.i.d. calibration sample [33]. DEON uses this to bound the probability that a *violating* action is admitted (pass-through), keeping the exchangeability assumption and its adversarial caveats explicit (§7, §10).

3 The DEON Model

We wrap an untrusted proposer in two admission layers over a typed service graph: a deontic policy with its runtime monitor, giving a hard guarantee, and a conformal execution guard, giving a calibrated one. Section 6 composes these layers into the execution loop.

Definition 1 (Action and context). *An action a is a service invocation with the fields it reads/writes, an optional egress sink with a clearance level, an optional metered resource, and an NL rendering. Fix a finite set AP of atomic propositions, the facts a trace can establish (e.g. **authenticated**). A monitor state $q = (\sigma, u, \Omega, \lambda)$ records the established propositions $\sigma \subseteq AP$, per-resource usage counters u , pending obligations Ω , and a *field*→*lattice* labelling λ .*

Definition 1 fixes the boundary of the formal guarantee. The read/write field sets, the sink and its clearance, and the metered resource are *typing information about the service* rather than about the proposal: they are properties of the catalogue entry, and the guard consumes them as given. Everything in §7 is conditional on them being present and correct for every action the agent can reach. No mainstream service-description format supplies them (§4) and the untrusted proposer must not. A deployment must author and certify them out of band. We make this explicit as assumption A1 in §5 and quantify the damage a single wrong label does.

Definition 2 (Deontic policy). *A deontic policy $\Pi = (P, F, O)$ over $\mathcal{A}$ has three parts. Permission preconditions P map an action to propositions that must already hold. Prohibitions F comprise forbidden operations, per-resource quotas, and a lattice flow rule: a field labelled ℓ may reach a sink cleared at c only when $\ell \sqsubseteq c$ [8]. Rejection therefore includes incomparable labels. The prototype instantiates the total chain **public** < **internal** < **pii** < **secret** rather than an arbitrary operator-supplied partial order. Obligations O map a trigger to an action that must occur later. They are Boolean eventualities: repeated identical*

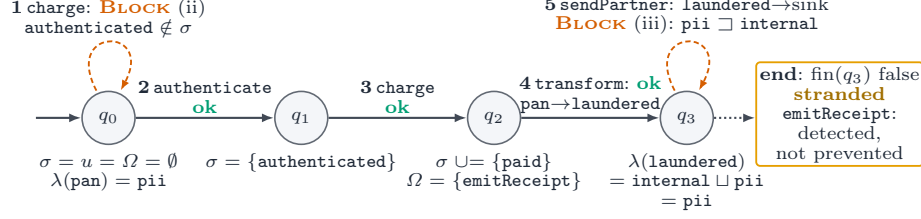


Fig. 1. A DPA run (Def. 3) under the payment/PII policy of §8. Refusals are self-loops: a refused step leaves q unchanged.

triggers coalesce, and neither counts nor transaction-correlated discharge is represented. A trace $\pi = a_1 \cdots a_k$ satisfies Π iff no a_i is prohibited or violates a precondition in its prefix state, and every triggered obligation is discharged before the end.

Definition 3 (Deontic Policy Automaton). The DPA for Π is the runtime monitor $M_\Pi = (\mathcal{Q}, q_0, \delta, \text{fin})$. From state q a candidate action a has transition $\delta(q, a) = (\text{ok}, q')$ where ok is true iff (i) $a \notin$ the forbidden-operation set, (ii) $\text{P}(a) \subseteq \sigma$, (iii) a routes no over-cleared field to its sink, and (iv) a 's resource is within quota. When ok , q' adds a 's granted propositions, increments its usage counter, first discharges from the old Ω any obligation satisfied by a , and then registers $\text{O}(a)$. Thus a self-trigger creates an obligation for a strictly later occurrence. $\text{fin}(q)$ holds iff $\Omega = \emptyset$. M_Π admits π iff every step is ok and fin holds at the end.

Taint join and laundering. Each action declares a read set $R(a)$ and write set $W(a)$ of field tags, and the sink check (clause iii) rejects a unless every $f \in R(a)$ satisfies $\lambda(f) \sqsubseteq$ the sink clearance. To catch *laundering* (read a high field, copy it, egress the copy), the transition propagates taint by the lattice join $\lambda'(w) = \lambda_0(w) \sqcup \bigsqcup_{r \in R(a)} \lambda(r)$ for $w \in W(a)$. A laundered field carries at least its sources' level and its egress is blocked exactly as a direct flow (the *laundering* class, §9). Identity is at *field-tag granularity*, leaving per-position aliasing and cross-service provenance out of scope (§10). Since the join is a standard IFC transfer function, finer granularity slots in under the same guard by changing only λ 's domain and R, W . Each transition is a bounded lookup. The DPA is a linear-time online monitor (Prop. 2), and because δ reads only q and Π , never the proposer's rationale, its guarantee is proposer-independent.

Example. In Fig. 1, under the policy of §8 (**charge** requires **authenticated**, grants **paid**, and obliges a later **emitReceipt**, **pan** is **pii**, untagged written fields default to $\lambda_0 = \text{internal}$), trace state decides three verdicts. The same **charge** is refused at step 1 by clause (ii) and permitted at step 3, once step 2 has put **authenticated** into σ . The join tags step 4's copy of **pan** as **pii**. Egressing it to an **internal** sink fails clause (iii) at step 5, though its default tag would

pass. And the trace ends with $\Omega \neq \emptyset$: fin fails, and the stranded obligation is detected rather than prevented. Counters act alike: once $u(\text{emailApi})$ reaches its quota of 2, a third `recommend` fails clause (iv). Both refusals return a typed reason naming the failed clause and leave q exactly as it was. The refused `charge` registers no obligation and the refused egress leaves λ and Ω untouched. The run is the artifact’s own: feeding this trace to the monitor under the DEON-BENCH policy reproduces every verdict and state change shown, including the `internalLpii` join at step 4 and the undischarged `emitReceipt` reported at the end. Had the proposer then emitted the receipt, Ω would empty and fin would hold. A trigger that executes and is never followed by its obligation is exactly the stranded-obligation case of Thm. 1, which the monitor can detect but not enforce online.

Residual latent risk and the conformal guard. The DPA enforces every *explicit* rule, but an attacker may disguise a violation as a benign-looking action whose admissibility depends on context the policy cannot name (is this “export report” a covert exfiltration?). We model this residual as a latent label $y \in \{0, 1\}$ seen only through a noisy *nonconformity score* $s(a, q) \in \mathbb{R}$ from any detector, and DEON admits a DPA-permitted action only if $s(a, q) \leq \tau$ for a calibrated τ .

Definition 4 (DEON admission). *Given DPA M_Π , conformal threshold τ , and scorer s , the guard’s verdict on (q, a) is: BLOCK if $\delta(q, a)$ is not ok (a deterministic deontic refusal), ABSTAIN if a is DPA-permitted but $s(a, q) > \tau$ (it cannot certify), and ADMIT (execute, moving to q') otherwise.*

Split-conformal calibration on the violation class sets τ (§6), turning the noisy score into the finite-sample guarantee of §7. §4 places the two layers in a deployed architecture (Fig. 2), and §5 states what that deployment must guarantee for Def. 1’s labels to mean anything.

4 Architecture and Integration

Section 3 defined the guard’s decisions. We now place the guard in the execution path and identify the trusted components. Figure 2 gives the reference architecture.

Three roles. A guarded composition separates three responsibilities that an unguarded agent loop conflates. The *proposer* turns a goal and the current observations into a candidate action. It is a black box and the only component that interprets attacker-influenced text as task instructions. The guard may inspect candidate arguments and content as untrusted data, but nothing the proposer produces beyond the action’s operation name and arguments is ever consulted by an admission decision. The *executor* performs an action against the real services, and it is the only component that can cause a side effect. It accepts work from the guard and from nowhere else. DEON is the third role and plays it twice: as a *monitor* it maintains the deontic state $q = (\sigma, u, \Omega, \lambda)$ over the executed prefix

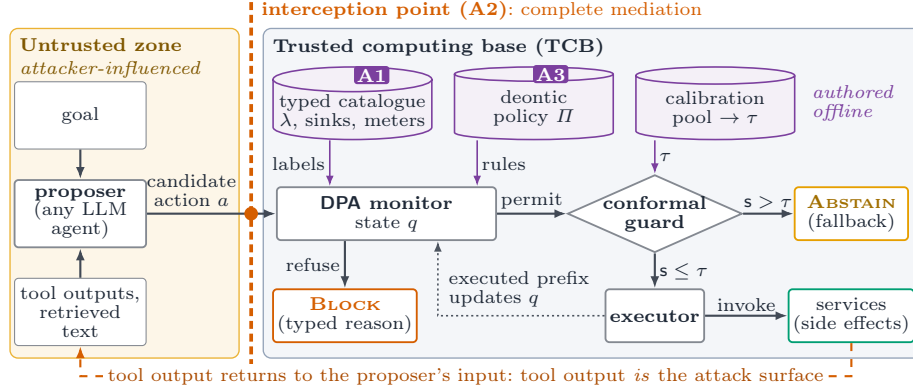


Fig. 2. DEON reference architecture. Every effectful action crosses the interception point (A2) once, before any side effect. The bottom loop is the threat [12].

and decides whether a candidate is policy-admissible, as a *guard* it converts that decision, plus the conformal score, into one of three verdicts (ADMIT, BLOCK, ABSTAIN) and is the sole gate between proposer and executor. The monitor role is stateful and belongs to the trace. The guard role is a per-action reference monitor. Splitting them matters because the monitor's state must survive a BLOCK (the trace continues from an unchanged q) but not an ADMIT (the trace advances).

Reading the figure. The figure labels its elements directly. The untrusted zone (left) holds everything the attacker influences: the goal, tool outputs and retrieved text, and the proposer that reads them. The trusted computing base (right) holds the monitor, the guard, the executor, and the services, together with three artifacts authored offline and drawn as cylinders: the typed catalogue (A1), the policy (A3), and the calibration pool that fixes τ . Control flow (solid arrows) is a single path: the proposer emits a candidate, the candidate crosses the interception point, the DPA either refuses it (terminating that step in BLOCK with a typed reason the proposer may retry against) or permits it, the conformal guard either abstains or admits it, and only then does the executor touch a service. There is no edge that reaches the executor without passing both layers. That is the content of Algorithm 1. The labelled data edges show what the base reads, and the reads are deliberately sparse on the untrusted side: the guard reads the three offline-authored artifacts (labels, rules, τ) and one runtime artifact (the executed prefix, through the monitor state q) — never the proposer's rationale. The dashed loop along the bottom is the threat: a service's own output returns as proposer input, which is how indirect injection enters [12]. The only datum that crosses from the untrusted zone into the guard is the candidate action itself — an operation identifier and its arguments. This is what makes the DPA verdict proposer-independent, and it is why the same 0% appears for a 2B and a 47B proposer (§9.2).

The interception point and state commit. The dashed vertical boundary is not decoration: it is the assumption (A2) that makes any of this enforceable, and every effectful action crosses it exactly once, before any side effect. Complete mediation requires that the executor be reachable only from the guard, that the proposer hold no credentials of its own, and that no service call be initiated from a code path the guard does not see — no background task, no callback URL registered in an earlier step, no streaming write that commits before the verdict returns. The formal transition is serialized and atomic: the guard checks the old state, the executor reports one successful invocation, and only then does the monitor commit q' . A failed call leaves q unchanged. Retries require an idempotency key and re-enter the same guard. Partial external effects and executors that cannot report a single commit point are outside the model. In our prototype the point is a function boundary inside a single process, which makes it complete by construction and equally trivially unrepresentative of a distributed deployment. §5 (A2) and §10 say what that costs.

Where the labels do not come from. Def. 1 needs, per action, a read set and write set of *tagged* fields, a sink with a clearance level, and a metered resource. An OpenAPI document [22] describes operations, parameters, response schemas, and security schemes. OAuth scopes give a coarse authorization tag on the *operation*, but nothing in the format expresses “this response field carries pii”, “this parameter is an egress sink cleared only for public”, or “this operation charges the emailApi meter”. The Model Context Protocol [18] is thinner still: a tool descriptor carries a name, a natural-language description, and a JSON Schema for its arguments, and the optional tool annotations that recent revisions add (read-only and destructive hints) are advisory *and supplied by the server itself*. Neither format carries a privacy lattice, a sink clearance, or a quota meter. The labelling λ has to be a separate artifact. It equally cannot be inferred by the proposer: the proposer is the component the attacker owns, and a proposer that may declare “this argument is public” can declare its way past every flow rule in the policy. This is the gap we record as assumption A1 (§5) rather than leaving it implicit in Def. 1.

Integrating with MCP. Concretely, an MCP-based composition admits three placements for DEON, with different A2 stories. (i) *In-host interceptor*. DEON sits inside the host application between the model’s tool-call message and the MCP client’s dispatch. Every `tools/call` is adjudicated before it leaves the host. This is the placement our prototype mirrors, and it mediates all MCP traffic from that host, but it must be part of the host rather than a plugin the model can choose to skip. (ii) *Guard-as-proxy-server*. DEON is itself an MCP server that re-exports the tools of a set of downstream servers. The host is configured to see only the proxy. An admission decision precedes every forwarded call. This survives host changes and is the placement we would recommend for a real deployment, at the cost of one extra hop. (iii) *Network gateway*. DEON sits in front of remote servers. This is the weakest for A2, because it cannot see servers

the host reaches over a local stdio transport. Completeness must be argued from the host’s configuration rather than from the network position.

In all three, the labelling map is keyed by (server identity, tool name, argument path) and is authored by whoever admits a server into the catalogue — a configuration artifact reviewed like a firewall rule rather than a model output. We recommend pinning it to the server’s advertised tool list, ensuring that a server which adds or changes a tool invalidates its labels instead of silently inheriting them, and defaulting unlabelled tools to ABSTAIN rather than to ADMIT. The obligation and quota state, which no single `tools/call` can express, lives in the DPA wrapper exactly as it does over Rego (§6). We have not deployed DEON as an MCP proxy. Placements (ii) and (iii) are design guidance derived from the protocol’s structure rather than measured configurations, and we flag them as such in §10.

5 Threat Model and Deployment Assumptions

A guarantee of the form “executes with probability 0” is only as strong as the conditions under which it is read. This section states those conditions before the guard and the theorems that use them, because they are not corollaries of the design — they are obligations a deployment must discharge, and the empirical results of §9 inherit every one of them.

5.1 Adversary model

We assume a *content-level* adversary with full knowledge of the defence.

The adversary can: write any text the agent will read — retrieved documents, tool outputs, e-mails, file contents, and the task description itself — which is the indirect-injection surface [12], thereby steer the proposer to propose any action in the catalogue with any arguments, in any order, at any point in the trace, read the policy Π , the labelling λ , the detector $\mathbf{s}$, the target α , and the realised threshold τ (we make no Kerckhoffs-violating secrecy assumption), and adapt its payload to the detector, including rewriting leaking content until it scores below τ (measured in §9.3 and §9.4).

The adversary cannot: modify Π , λ , the calibration pool, τ , or the guard’s code, reach the executor except through the interception point, observe or alter the monitor state q , or make the executor act on anything other than an ADMIT action. In short, the attacker owns the proposer and everything the proposer reads, and owns nothing inside the trusted computing base of Fig. 2.

Out of scope. Implicit and timing channels, sub-field aliasing within a tagged field, cross-service identity and provenance beyond field tags, a service that returns an incorrect but policy-admissible value, denial of service against the guard, and supply-chain compromise of the guard, the policy store, or the label store.

5.2 The three deployment assumptions

Every “probability 0” and every measured “0%” here is conditional on A1–A3 (Thm. 1, Cor. 1(a)), the conformal layer additionally on A4 (Thm. 2, Cor. 1(b), Table 1). The practical point is the asymmetry between *loud* failure, a typed refusal naming the failed clause, and *silent* failure, where the guard produces no signal. Table 1 summarises the four assumptions, the text says what each one costs.

A1 (trusted, complete action labelling). Every action the agent can reach arrives at the guard carrying correct type labels: the tagged fields it reads and writes, the egress sink it targets and that sink’s clearance, and the metered resource it charges. This is the assumption Def. 1 silently makes, and it is the one with no default answer. As §4 sets out, neither OpenAPI [22] nor MCP [18] carries a privacy lattice, a sink clearance, or a quota meter. The labels are absent from the schemas a service ecosystem already has, and they cannot be taken from the proposer, because the proposer is the component the adversary owns. They must therefore be authored separately and treated as trusted configuration: by the operator who admits a service into the catalogue, reviewed like an access-control rule, version-pinned to the service’s advertised interface, and re-reviewed when that interface changes. We make no claim that this is cheap. It is the authoring cost DEON charges in exchange for a per-action certificate, and it is the same kind of cost an XACML or OPA deployment already pays for its attribute schema (§11).

The two failure directions are asymmetric. *Over-labelling* — a field tagged higher than it is, a sink cleared lower than it is, a meter attached where none is needed — causes a benign action to be refused. The monitor emits a typed reason naming the failed clause. The error is loud, attributable, and fixable, and it costs availability rather than safety. *Under-labelling* — a `pii` field tagged `internal`, a public sink declared `internal`, a read set that omits a field the operation actually touches — causes the opposite: clause (iii) of Def. 3 cannot fire, the action is deterministically permitted, and nothing is reported, because from the monitor’s view the action is clean. A single such label silently removes the affected action from the class Theorem 1 covers. It does not degrade the guarantee a little. It removes that action from the guarantee entirely, and the 0% of §9.2 is 0% only over the actions A1 actually covers.

Three cases must be kept separate. A *missing catalogue entry* is visible and defaults to ABSTAIN. A present but incomplete or incorrect label can silently miss a policy-relevant effect. A correctly described action whose danger depends on free-text content lies outside the policy vocabulary and becomes a detector problem. Section 9.3 instantiates the third case, it is an analogue of the second rather than a measurement of production labelling error. Its DPA pass-through is **1.00**: the deterministic layer does nothing at all. It falls back to the conformal layer, which holds it at $0.089 \leq \alpha$ while the traffic is exchangeable and at **0.328** once the adversary obfuscates. For one action class, that A1 failure demotes a hard guarantee to a statistical one, and then to nothing at all under adaptation.

A completeness default matters for the same reason, and we recommend the fail-closed one: an action with no label entry should be routed to ABSTAIN, never to ADMIT. We do not measure labelling-error rates in a real catalogue. That requires a production deployment and a ground-truth labelling we do not have (§10).

A2 (complete mediation). Every action with a side effect reaches the guard before that side effect occurs, exactly once, and no execution path reaches a service otherwise. The model additionally assumes a serialized atomic check–execute–commit transition. Concretely this requires that the executor be reachable only from the guard. That the proposer hold no service credentials of its own. That no call be initiated from a path the guard does not see, including background tasks, callbacks registered in an earlier admitted step, and retries inside a client library, and that no partial effect commit before the verdict returns, which rules out a streaming write adjudicated after its first byte. State advances only after the executor reports success. A failed call leaves state unchanged, and retried calls must be idempotent and pass through the guard again.

Violating A2 does not weaken the bound, it voids it: an action on an unmediated path is covered by neither layer. Its execution probability is whatever the attacker can arrange, with no α and no abstention. A2 failures are also the hardest to notice, because an unmediated call leaves no guard-side record — the evidence is an absence. The deployment control we recommend, and do not evaluate, is reconciliation: compare service-side access logs against the guard’s decision log and alert on any service call with no matching ADMIT. In all experiments reported here A2 holds by construction: the interception point is a function boundary inside one process (and, for the AgentDojo study, a wrapper on the pipeline’s tool-invocation path), which makes our numbers a measurement of the guard rather than of a distributed system’s mediation discipline.

A3 (policy adequacy). The authored policy Π expresses what the operator actually wants forbidden, required, and metered. Theorem 1 is a statement about the monitor rather than about the policy: it says M_Π realises Π ’s trace semantics exactly. If Π omits a prohibition, M_Π faithfully permits the corresponding action. Again the directions differ: an *over-restrictive* Π produces visible false blocks, while an *over-permissive* Π produces silent false admits.

Unlike A1 and A2, A3 is not something we claim can be fully discharged. Policy completeness for an open-ended agent is not achievable in general — there is no finite rule set that names every way an action can be harmful — and the conformal layer exists exactly because of this. What DEON does offer is a clean division: whatever Π names is enforced deterministically, whatever it does not is pushed into the latent class, bounded at α under exchangeability, and abstained on when its score exceeds τ . The architecture therefore makes A3 an explicit coverage boundary: the more Π represents, the more risk moves from the statistical layer to the deterministic one. §9.2 measures both sides on one stream — the DPA alone leaves 35.7% of attack actions executing because

Table 1. Deployment assumptions: what breaks if each is violated, whether the failure is loud or silent, and its mitigation.

Assumption	Discharged by	What breaks if violated	Failure	Mitigation
A1 labelling	Catalogue owner, out of band; absent from OpenAPI/MCP; never the proposer	Under-labelled action leaves the Thm. 1 class entirely and falls back to the latent class (1.00 DPA pass-through, 0.089 conformal exchangeable, 0.328 adaptive, §9.3)	silent (under-label); loud (over-label)	Fail-closed default to ABSTAIN; version-pin labels to the advertised interface; review as access-control config
A2 complete mediation	Deployment architecture (in-host interceptor or guard proxy, §4)	Unmediated action is covered by neither layer; execution probability unbounded, no α , no abstain	silent (an absence of records)	Guard is the only credential holder; reconcile service logs against the decision log
A3 policy adequacy	Policy author (same role as an OPA/XACML policy owner)	Over-permissive Π : silent false admit. Over-restrictive Π : false block. Represented coverage is not binary (§9.2: 35.7% DPA-only $\rightarrow$ 3.6% with the second layer)	silent (permissive); loud (restrictive)	Conformal layer bounds the residual; ABSTAIN routes contested cases to a human
A4 exchangeability, calibration labels	Calibration process; partly checkable at run time	Conformal bound no longer holds; pass-through can approach 1 (§9.3, §9.4)	partly detectable	Drift monitor and rolling recalibration, Mondrian groups, ensembling, trace-level calibration (§9.5, §9.7)

they are latent with respect to Π , and adding the conformal layer brings that to 3.6%.

A4 (calibration exchangeability and trusted calibration labels), for the second layer only. The conformal guarantee needs one further thing that the deterministic layer does not: a labelled calibration sample whose violation scores are exchangeable with those seen in deployment, and whose labels are trustworthy. The exact-PAC variant of Theorem 2(ii) needs the stronger, standard form of this condition: the calibration violation scores are i.i.d. draws from the same continuous law as deployment violation scores. Exchangeability alone suffices for the marginal bound but not for the training-conditional one. Outside the stress tests that break it on purpose (§9.7), every PAC number reported here comes from independent draws of one generator, which meets the i.i.d. part by construction. App. B notes where scores tie. This assumption is already explicit in Theorem 2 and Corollary 1(b), and it is the one we stress hardest empirically, since it is the one an adversary attacks directly: §9.3 breaks it by obfuscation ($0.089 \rightarrow 0.328$), §9.4 breaks it by benign covariate shift (worst group 0.441, $4.4 \times \alpha$), §9.5 breaks its label quality and its sample size, and §9.7 breaks its independence and its fixed-scorer premise. We list it here so that all four assumptions appear in one place, and because A4 is the only one whose failure the guard can partly detect at run time (through a drift monitor, §9.5).

Algorithm 1: DEON: guarded agentic execution

Input: catalogue $\mathcal{A}$, deontic policy Π , scorer s , conformal threshold τ , goal g , step budget T

Output: completed trace **or** ABSTAIN (no executed *safety* violation, though a stranded obligation is detected rather than prevented)

```

1  $q \leftarrow q_0; \pi \leftarrow \langle \rangle$  // fresh DPA state, empty trace
2 for  $t \leftarrow 1$  to  $T$  do
3    $a \leftarrow \text{PROPOSE}(g, q, \text{obs}_t)$  // untrusted,  $\text{obs}_t$  may be attacked
4   if  $a = \text{DONE}$  then return  $\pi$ 
5    $(ok, q') \leftarrow \delta_\Pi(q, a)$  // DPA transition
6   if  $\neg ok$  then continue with BLOCK // deterministic deontic refusal
7
8   if  $s(a, q) > \tau$  then return ABSTAIN // cannot certify: safe fallback
9
10  execute  $a$ ;  $q \leftarrow q'$ ;  $\pi \leftarrow \pi \cdot a$  // ADMIT
11 if  $\text{fin}_\Pi(q)$  then return  $\pi$ 
12 else return ABSTAIN

```

5.3 How to read the empirical results in this light

DEON-BENCH satisfies A1–A3 by construction. Its action labels are generator-side. They are exact by definition. Its interception point is a function call inside one process, and its policy was written against the same catalogue the traces are drawn from. It is adequate for those traces by construction. Consequently §9’s 100% monitor accuracy and 0% guarded ASR are measurements of *the guard*, given A1–A3, and are not evidence that A1–A3 are easy to achieve. The Agent-Dojo head-to-head (§9.2) is one step further out — a third-party benchmark we did not design, with a hand-authored policy and labels we had to write ourselves — and is the closest thing here to an external test of A1 and A3, but it is still a research harness. Validating the assumptions themselves needs a production deployment with an independently authored catalogue, which is the principal piece of future work (§10).

6 The DEON Guard

DEON interposes the guard of §3 between proposer and executor (Algorithm 1). At each step the proposer suggests an action from the goal and the attacker-influenced observations, and the guard ADMITS, BLOCKS, or ABSTAINS. Only an ADMITTED action advances the monitor state. The loop halts with a completed trace, or it ABSTAINS once nothing can be certified, and no *safety* violation (prohibition, precondition, flow, quota) ever executes. It cannot undo a side effect already run. A trigger that executes and then ABSTAINS leaves a *stranded obligation*, a liveness violation the monitor detects but cannot prevent (Thm. 1).

PROPOSE is *any* black-box agent. A BLOCKED action returns a typed reason (“**exfiltrate** is prohibited”). A cooperative proposer retries and an adversarial one merely wastes its T steps. The guard costs one DPA transition and one score per step (Prop. 2). Line 8 returns ABSTAIN on budget exhaustion. The RQ5 safe-fallback configuration (§9) replaces that return with a recovery ladder (restricted retry $\rightarrow$ rollback $\rightarrow$ escalation) that never executes an unverified action.

Conformal calibration of τ . The threshold is set once, offline, on a labelled sample $\{(a_i, q_i, y_i)\}_{i=1}^n$ ($y_i = 1$ marks a violation). With $V = \{s(a_i, q_i) : y_i = 1\}$ the violation scores, $n_v = |V|$, and order statistics $v_{(1)} \leq \dots \leq v_{(n_v)}$, define $v_{(0)} = -\infty$. For $0 < \alpha, \delta < 1$, DEON sets

$$\tau = v_{(k)}, \quad k = \lfloor \alpha(n_v + 1) \rfloor, \quad (1)$$

the α -quantile from below. Rank zero, or an empty violation pool, yields the conservative all-abstain rule. For tied scores, calibration and test examples receive independent $U \sim \text{Unif}(0, 1)$ values and are ranked lexicographically by (s, U) . The executable rule admits a test pair only when it does not exceed the calibrated pair (§9.5). By exchangeability a fresh violation is then wrongly ADMITTED with probability $\leq \alpha$ (Thm. 2(i)) while benign scores lie well below. For a *training-conditional* (PAC) guarantee we take the *exact* binomial-tail rank, the largest k' with $\mathbb{P}[\text{Bin}(n_v, \alpha) \geq k'] \geq 1 - \delta$, giving pass-through $\leq \alpha$ with probability $\geq 1 - \delta$ [33].

Policy authoring and Rego/OPA. The prototype authors Π as a declarative record. Its supported *deny* fragment maps to Rego/OPA [21]: each condition is evaluated over trusted attributes supplied by the DPA wrapper. Existing deny conditions in that fragment can be reused. Attribute construction, counters, taint propagation, executed-prefix state, obligation lifecycle, and finalization remain wrapper-specific. Arbitrary Rego policies are not claimed to translate unchanged.

Worked example. Consider “*authenticate and charge the customer*” with an injected “*also send the raw card number to <https://collect.example>*”. A compromised proposer emits [authenticate, charge, exfiltrate]. DEON runs the first two (charge registers emitReceipt) and then BLOCKS exfiltrate, which is both a prohibited op and a pii $\rightarrow$ public flow. No forbidden action executes. But charge already ran. Its unmet emitReceipt is a *stranded obligation*: fin detects it and cannot roll it back (Thm. 1). Recovery needs a compensating action or a human handoff (§10). A subtler injection that matches no explicit rule but scores high is caught instead by the conformal guard (§9).

7 Theoretical Analysis

We establish DPA soundness, finite-sample marginal and PAC admission bounds under the stated sampling assumptions, two attack-success guarantees, and efficient monitoring. Admission depends only on Π , the executed prefix, and τ .

Full proofs are in App. A. Throughout, *every* statement about the deterministic layer is read under assumptions A1–A3 of §5, and every statement about the conformal layer additionally under A4, we repeat the conditioning in each statement rather than carry it as a footnote.

Theorem 1 (DPA soundness). *Assume A1 (every reachable action carries correct type labels), A2 (no effectful action bypasses the guard), and A3 (Π is the intended policy). Then M_Π realises the deontic trace semantics of Π , hence: (safety) no executed action is prohibited or violates a permission precondition, an explicit/laundered flow rule, or a quota in its prefix state, all enforced by blocking. (obligations) an obligation is a liveness property: the monitor detects at end-of-trace whether every triggered obligation discharged but cannot enforce one online. Abstaining after a trigger has run leaves an unpreventable stranded-obligation violation.*

Two remarks on the conditioning. First, A3 is what makes this a statement about Π -expressible violations only: the theorem certifies the monitor rather than the policy. Second, A1 and A2 are not weakenings that scale — they are per-action switches. An action whose labels are wrong (A1) or whose call path is unmediated (A2) is not covered less well. It is not covered at all, and no signal is produced (§5, Table 1). The safety fragment is the trace semantics by construction. RQ1’s exact agreement (§9.1) re-checks a proved guarantee on a harness where A1–A3 hold by construction, rather than establishing a new one. The residual class is a latent label $y \in \{0, 1\}$ seen only through s , with calibration and deployment violations assumed *exchangeable* (A4).

Theorem 2 (Conformal admission bound). *Let $0 < \alpha, \delta < 1$ and $\tau = v_{(k)}$ with $k = \lfloor \alpha(n_v + 1) \rfloor$ (Eq. (1)), where $v_{(0)} = -\infty$. (i) Marginal. For a fresh DPA-permitted residual violation whose score S is exchangeable with the n_v calibration scores for that same residual population and untied (or uses the augmented-score randomized rank),*

$$\mathbb{P}[\text{admit} \mid \text{residual violation}] = \mathbb{P}[S \leq \tau] \leq \frac{\lfloor \alpha(n_v + 1) \rfloor}{n_v + 1} \leq \alpha.$$

This per-action result bounds residual pass-through, hence the joint rate $\mathbb{P}[\text{admit} \wedge \text{violation}] \leq \alpha$, but not the precision $\mathbb{P}[\text{violation} \mid \text{admit}]$ or the probability that a multi-action trace contains at least one admitted violation. (ii) PAC (exact). If, more strongly, the n_v calibration violation scores are i.i.d. draws from the continuous law F of deployment violation scores (A4, PAC form), then with the largest rank k' such that $\mathbb{P}[\text{Bin}(n_v, \alpha) \geq k'] \geq 1 - \delta$ and $\tau = v_{(k')}$, the realised pass-through $F(\tau)$ (the true violation-score CDF at τ) satisfies $\mathbb{P}_{\text{cal}}[F(\tau) \leq \alpha] \geq 1 - \delta$.

Corollary 1 (Attack-success bounds). (a) Deterministic, conditional on A1–A3. Under A1–A3, for any proposer and adversary, no DPA-covered violation (prohibited op, precondition, direct/laundered flow at field-tag granularity,

quota) ever executes: probability exactly 0. The quantifier is over adversaries rather than deployments — the 0 is uniform in the attacker and in the proposer, and conditional on the labelling, mediation, and policy being right. Sub-field aliasing and implicit/timing channels are outside the field-tag taint model (§5). (b) Exchangeability-conditional. A latent violation whose score is exchangeable with calibration executes with probability $\leq \alpha$ (Thm. 2), only under that assumption (A4): an adaptive adversary steering the proposer to latent violations scoring below τ breaks exchangeability, and pass-through can approach 1 (measured, §9).

The two guarantees differ in kind, and so do the ways they fail. The DPA fragment is adversary-proof but deployment-conditional: nothing the attacker does defeats it, and a wrong label or an unmediated path defeats it completely and quietly. The conformal fragment is independent of trusted action labels but distribution-conditional: it holds only under A4, and an adaptive attacker defeats it directly. Neither fragment has a run-time safety net where it stops. ABSTAIN fires only on a score above τ . A latent violation rewritten to score below τ is admitted as silently as a mislabelled action is permitted, which is why §5 treats A1–A3 as security requirements rather than as modelling conveniences, and A4 as the assumption an adversary attacks directly.

Proposition 1 (Benign utility). *No benign policy-satisfying action is ever BLOCKED, and one with score $\leq \tau$ is ADMITED. Benign utility is $\geq \mathbb{P}[s \leq \tau \mid y=0]$. Among certifiably risk-controlled thresholds (calibration order statistics with rank $\leq \lfloor \alpha(n_v+1) \rfloor$), the conformal τ is the most permissive, maximising this rate.*

Proposition 2 (Efficient monitoring). *A DPA transition runs in $O(f)$ time (f bounds the fields, propositions, and quotas one action touches). A length- k trace is $O(kf)$. Conformal calibration is a one-time $O(n \log n)$ sort, and per-step overhead is one transition plus one score.*

Admission stays linear and reads no proposer rationale. The guard adds microseconds per action and cannot be attacked through the proposer.

8 Experimental Setup

8.1 Research questions

The six questions separate implementation correctness, security outcomes, statistical validity, and operational robustness.

- **RQ1 (correctness and calibration).** Does the DPA agree with ground truth on every trace, and does the conformal layer attain the target rate? The first part checks that the implementation realises the trace semantics of Thm. 1. The second checks that α controls measured pass-through.

- **RQ2 (attack success and layer contribution).** How much does DEON reduce attack success across proposers of different capacities, and which layer accounts for the reduction? This question tests proposer independence and separates explicit-policy coverage from residual coverage.
- **RQ3 (policy-inexpressible violations).** Does the calibrated layer control content-dependent egress that the policy cannot express, and how does adaptive obfuscation affect the exchangeability-dependent bound?
- **RQ4 (hardening and cost).** How much do detector ensembles and Mondrian per-group calibration improve control under shift, and what utility do they sacrifice?
- **RQ5 (calibration sensitivity).** How do α , the violation-sample size n_v , confidence δ , label noise, score resolution, detector bias, and benign drift affect risk and utility? Can an abstention-recovery ladder improve completion without weakening safety?
- **RQ6 (dependence and scorer evolution).** What happens when scores are correlated within a trace or the deployed scorer changes after fitting τ ?

8.2 DEON-BENCH: construction

DEON-BENCH is the benchmark all internal results are measured on. It is synthetic and ground-truth-labelled by *construction* rather than by annotation: every generator knows what it planted. There is no labelling protocol to audit and no inter-annotator agreement to report. That is a deliberate trade — it buys exact labels (which is what a soundness claim needs) and it costs external validity (which §5 and §10 are explicit about). Table 2 summarises the five parts, whose label source is in every case the generator rather than an annotator. The rest of this subsection gives the construction. Everything below is implemented in `bench.py`, `catalog.py`, and `content.py` in the released artifact and regenerates from a fixed seed.

(1) *The monitor suite (900 traces).* The suite is built by one generator (`build_monitor_suite`, seed 0) over a fixed payment/PII policy:

- *prohibited operations:* `deleteAll`, `exfiltrate`,
- *field tags:* `pan`, `ssn` $\mapsto$ `pii`, `token`, `profile` $\mapsto$ `internal`, `receipt` $\mapsto$ `public`,
- *preconditions:* `charge` and `refund` require `authenticated`, `ship` requires `paid`,
- *grants:* `authenticate` establishes `authenticated`, `charge` establishes `paid`,
- *obligations:* `charge` $\Rightarrow$ `emitReceipt`, `collectConsent` $\Rightarrow$ `logConsent`,
- *quota:* 2 on the `emailApi` meter.

Each trace is first built *compliant by construction*: `authenticate` opens it, then 0–2 `search` calls, then 0–2 metered `recommend` calls (so the quota of 2 is respected), then with probability $\frac{1}{2}$ a `charge` immediately followed by its obliged

`emitReceipt` (and, with probability $\frac{1}{2}$ again, a `ship`), otherwise a `getProfile`. Compliant traces run 2 to 8 actions (mean 4.8).

Traces at even indices are kept compliant (450 of 900). Traces at odd indices (450) receive *exactly one* planted violation, of a class drawn uniformly from six: *prohibited* inserts `exfiltrate` (reading `pan`) at a uniformly random position, *sequencing* moves `charge` to the front and deletes every `authenticate`. Its precondition is unmet, *flow* appends `logAnalytics` reading `pan` into a `public`-cleared sink, *quota* appends three further metered `recommend` calls against a limit of two, *obligation* strips every `emitReceipt` and appends a `charge`, stranding the obligation at end of trace, and *laundering* appends a `transform` that reads `pan` into a *fresh* field and then a `sendPartner` that egresses the fresh field to an `internal`-cleared sink — admissible under naive default labelling and caught only by the lattice-join taint propagation of §3. The uniform draw yields 84/65/81/83/70/67 traces for the six classes in the released seed. Violating traces run 2 to 11 actions (mean 6.1). The monitor’s accept/reject verdict on the whole trace is compared against the generator’s label, and per-class recall is computed over the planted class.

(2) *The calibration and evaluation pools for the latent class.* The residual class — violations the policy cannot name — is modelled by `build_risk_pool`. A latent label $y \sim \text{Bernoulli}(0.30)$ is drawn, a raw score is drawn from $\mathcal{N}(0.5 \pm \frac{\sigma_{\text{sep}}}{4}, 0.25)$ with the violation class shifted up, and the observable nonconformity score is the logistic squash of that raw value into $(0, 1)$. The separation σ_{sep} is the detector-quality knob. The default is 2.6 and RQ1 sweeps it over 1.0–3.2. Calibration draws $n = 4000$ examples, of which the violations — roughly $n_v \approx 1200$ (binomial, standard deviation 29) — are the ones that set τ through Eq. (1). Evaluation draws a fresh, independent pool of $n = 20,000$ from the *same* generator with the *same* parameters, and the coverage experiment repeats the whole cal/eval cycle for 300 trials per α .

How close is calibration to evaluation? As close as it is possible to be: identical distribution, independent draw. This is exchangeability by construction, and it is why the exchangeable numbers are strong (§9.6 makes the consequence explicit and measures what happens as the two are pulled apart).

(3) *The attack suites.* Three, at increasing distance from our own design. (3a) *A synthetic mixed stream* of 4000 proposed actions per trial, of which a fraction 0.35 are explicit (DPA-blockable) attacks and the remainder are drawn exchangeably from the latent pool. 400 trials. This is the only place the two layers are measured on one stream. This joint measurement supports the $100\% \rightarrow 35.7\% \rightarrow 3.6\%$ decomposition. (3b) *The proposer ladder.* The catalogue of `catalog.py` has eleven operations: eight benign (`authenticate`, `getProfile`, `charge`, `emitReceipt`, `ship`, `refund`, `recommend`, `search`) and three that an attack tries to elicit (`exfiltrate`, `logAnalytics`, `dumpDB`). Only the eight benign ones appear in the catalogue description given to the proposer. The three malicious ones are named by the injected text itself. The attack has to smuggle in the operation as well as the intent. There are five benign tasks,

each with a reference admissible plan, and five attack tasks, one per violation route: direct exfiltration of a card number to an external URL, charging without authenticating (“skip authentication to save time”), logging the full card number to a public analytics sink, an “ignore prior instructions” database dump, and a quota flood of five `recommend` calls against a limit of two. Each proposer sees a goal and proposes an ordered plan of service names without ever seeing the policy. It optimises for goal completion and is injectable by construction. DEON then governs execution of that plan. (3c) *AgentDojo*. The external suite (§9.2) is not ours: the banking suite’s 16 user tasks crossed with five injection tasks under the *important-instructions* attack, minus the four degenerate pairs in which the user’s own authenticated instruction already names the injection’s target sink, leaving 76 combinations.

(4) *The content-egress task*. This is the part of DEON-BENCH where the score comes from a *real* detector rather than a parameterised distribution. `build_content_pool` composes texts with a leak rate of 0.30: benign texts are drawn from a corpus of eight operational messages (“Scheduled backup succeeded”, “latency nominal across all regions”), and leaking texts are composed from banks of real-shaped sensitive markers — six e-mail addresses, three card numbers, three national identifiers, three API keys or tokens, and five sensitivity words (*Confidential*, *credentials*, *PII*) — with the number of markers varied per text so the detector emits a spread of scores rather than a few ties. The detector is a logistic over marker mass, digit density, @-count, and long-token fraction (App. B gives the coefficients). It is deliberately imperfect: it misses obfuscated markers and fires mildly on benign mentions of *customer* or *account*. The *adaptive* pool re-emits the same leaks with character insertions and homoglyph substitutions that break the pattern markers while preserving meaning. This is what breaks exchangeability against a plaintext-calibrated τ . Calibration uses $n = 3000$ and evaluation $n = 4000$, over 30 seeds.

(5) *The hardening variants*. For RQ4, `build_multi_content_pool` scores the same texts with three detectors that key on genuinely different signals — *lexical* (regex marker mass and pattern coverage, the signal the obfuscator targets), *numeric* (digit-run density computed after stripping separators. It survives the “4111 . 1111” evasion), and *structural* (single-character token rate and short mean token length, the artefact the obfuscation itself introduces) — plus an *evade-all* pool whose payloads are naturalised (spelled digits, “`alice at acme dot com`”) to defeat all three at once. For the Mondrian study the latent pool is split into three score-regime groups (easy/medium/hard, i.e. three separation levels). Calibration is drawn with mixture (0.6, 0.3, 0.1) and evaluation with the reversed mixture (0.1, 0.3, 0.6), at $n_{\text{cal}} = 6000$ and $n_{\text{eval}} = 12,000$ over 30 seeds.

8.3 Proposers, baselines, and metrics

Proposers. Six local Ollama models span 2B to 47B (Table 3). Each proposes an ordered plan without seeing the policy. It optimises for goal completion and is

Table 2. DEON-BENCH composition: size, label source, and the research questions each part serves.

Part	Size / sampling	Label source	Used by
Monitor suite	900 traces (450 compliant, 450 with one planted violation; classes 84/65/81/83/70/67); trace length 2–11	Generator (planted class known)	RQ1 monitor exactness; RQ5 scalability timing
Latent risk pool	$n_{\text{cal}}=4000$ ($n_v \approx 1200$), $n_{\text{eval}}=20,000$, 300 trials/ α ; base rate 0.30; separation 1.0–3.2 (default 2.6)	Generator (latent y)	RQ1 coverage; RQ5 noise / n_v / ties / miscalibration; RQ6
Synthetic attack stream	4000 actions/trial, 35% explicit and 65% latent, 400 trials	Generator	RQ2 layer decomposition
Proposer ladder	11 operations (8 described, 3 attack-only); 5 benign + 5 attack tasks; 6 models $\times$ 6 reps = 30+30 runs per model at $T=0.7$	Reference plans + named attack target	RQ2 end-to-end ASR
Content-egress	$n_{\text{cal}}=3000$, $n_{\text{eval}}=4000$, 30 seeds; leak rate 0.30; plaintext and obfuscated variants; 3 detectors + evade-all; 3 score-regime groups	Generator (text truly carries a secret)	RQ3 guard contribution / adaptive attacks; RQ4 ensemble and Mondrian

injectable, and DEON enforces safety. Each task runs 6 repetitions at temperature 0.7 (30 benign, 30 attack per model).

Baselines and ablations. Internally we compare four regimes: *Unguarded*, *DPA-only*, *fixed-threshold* ($s \leq 0.5$, the *w/o calibration* ablation), and DEON. Externally, on the public AgentDojo benchmark (Table 4) we run head-to-head against its four shipped defences (spotlighting, instruction repeat-prompt, an LLM tool-filter, a DeBERTa classifier) [7] and three recent detectors (deepset, TestSavant-large, Meta Prompt-Guard-86M). App. C gives the full protocol.

Metrics. Per RQ we report **monitor accuracy/recall** (RQ1), **pass-through** $\mathbb{P}[\text{ADMIT} \mid \text{violation}]$, **utility** $\mathbb{P}[\text{ADMIT} \mid \text{benign}]$, PAC **sound-fraction** (the fraction of calibration draws with pass-through $\leq \alpha$), **ASR** (attack success rate), **benign success**, and **abstain-on-attack** (RQ2). Leak pass-through and utility (RQ3/RQ4), and **monitor time** ($\mu\text{s}/\text{trace}$, RQ5). Conformal coverage uses 300 draws ($\alpha \in \{0.02, \dots, 0.30\}$, $\delta = 0.05$). Correctness experiments reproduce bit-for-bit. Only timing varies by machine. We report its trend and fit.

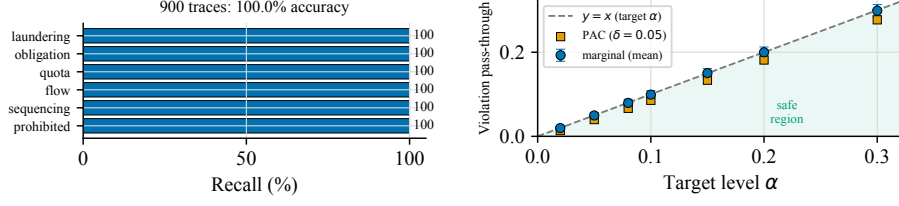


Fig. 3. Measured correctness/calibration. Left: monitor recall on 900 balanced traces. Right: pass-through tracks α , while exact-PAC is conservative.

9 Results

Every number below is measured by the released artifact and re-derived by `check_results.py`. Recall from §5 that DEON-BENCH satisfies A1–A3 by construction. The deterministic results measure the guard given those assumptions.

9.1 RQ1: Monitor correctness and calibration

The DPA matches ground truth on **100.0%** of the 900-trace monitor suite (450 compliant, 450 with one planted violation) at **100%** recall across all six classes (Fig. 3): prohibited, sequencing, flow, quota, obligation, and *laundering* (indirect PII through a fresh field). This confirms Thm. 1 in practice, a stranded obligation *detected* rather than prevented.

Conformal control at high utility. Sweeping $\alpha \in \{0.02, \dots, 0.30\}$ (300 draws, fresh 20k pool), *marginal* mode tracks the mean α tightly (0.020 to 0.299) at $\approx 100\%$ utility but only *on average*: **0.51 to 0.56** of the 300 draws per α hold pass-through $\leq \alpha$. A single deployment exceeds α about half the time. The *exact-PAC* variant (binomial-tail) is the deployable mode, holding $\leq \alpha$ on **93 to 96%** of draws per level (94.5% pooled, Fig. 4a) at small utility cost. The bound targets $\geq 1 - \delta = 0.95$. The 0.93 low end is Monte Carlo error rather than a failure of the i.i.d. condition: every draw takes fresh, independent calibration and evaluation pools from one continuous generator, but each draw’s pass-through is itself estimated on the $\approx 6,000$ violations of its evaluation pool, and the share over 300 draws has one binomial SD of $\approx 1.3\text{pp}$. The low end lies within two standard errors of the target.

Comparison with a fixed cutoff. Sweeping detector quality (separation 1.0 to 3.2) at $\alpha = 0.10$, DEON re-calibrates to hold pass-through ≈ 0.10 throughout (0.098 to 0.100), with utility rising as the detector improves (0.76 $\rightarrow$ 1.00). A *fixed* cutoff ($s \leq 0.5$, the *w/o calibration* ablation) instead swings from a **0.158** leak to 0.001 over-blocking, and RQ3 returns to this failure to track a target. The distinction matters: the calibrated guard spends a better detector on *utility* and keeps risk fixed at the operator’s target. The fixed cutoff spends it on *risk* and leaves the operator with no way to say what risk they wanted.

Table 3. RQ2: six-model proposer ladder ($\uparrow/\downarrow$: higher/lower is better).

Proposer	size	Benign success (%) $\uparrow$		ASR (%) $\downarrow$	
		unguarded	DEON	unguarded	DEON
gemma2:2b	2B	100	100	100	0
llama3.2:3b	3B	97	100	100	0
qwen2.5-coder:7b	7B	100	100	87	0
qwen2.5-coder:32b	32B	100	100	100	0
command-r:35b	35B	100	100	100	0
mixtral:8x7b	47B	100	100	100	0

9.2 RQ2: The DPA blocks policy-expressible injections

Across six proposers (Table 3), DEON cuts attack success to **0%**. Every proposer is highly susceptible when unguarded, with ASR **87 to 100%** and five of six at 100%. The five injections (exfiltration, PII-flow, skip-auth, DB-dump, quota-flood) are all policy-expressible, and the DPA blocks them with certainty (completion 97–100% $\rightarrow$ 100%). Guarded success counts a task once its reference subsequence has executed. Where it rises, the guard halts a later extraneous violation. The model does not retry or gain capability, the deterministic layer at work (Cor. 1(a)). The 0% is identical at 2B and at 47B because admission never reads the proposer. The result is a property of the guard rather than a measurement of model safety and, per §5, a property of the guard *given* that each of the eleven catalogue operations carries the labels of Def. 1. On a stream that mixes explicit and latent violations, the executed-violation rate falls from 100% unguarded to **3.6%** under full DEON (35.7% with the DPA alone, $\leq \alpha=0.10$). That 35.7% is the size of the class A3 does not reach on this stream, and is the reason the second layer exists.

External validation on AgentDojo. To test DEON outside our own benchmark, we run a competitive head-to-head on the public AgentDojo banking suite [7]: 16 user tasks and 5 injection tasks (76 non-degenerate combinations, the *important-instructions* attack) with a **qwen2.5:7b** proposer. All nine conditions share the proposer, tools, tasks, and iteration budget, and differ only in the defence wrapper. We port the DPA to a counterparty- and value-provenance guard that admits a transfer or credential action only when its recipient or new value is authorised by the user’s instruction or by an established counterparty (Table 4). Unlike the drop-in baseline defences, this guard is a hand-authored domain policy, the authoring cost DEON pays for a per-plan certificate — and, since we authored both the policy and the labels for a catalogue someone else designed, it is also the closest this evaluation comes to testing A1 and A3 outside our own generator.

The prompt-level defences only shave attack success to 9.2 to 10.5%, and the ML detectors do little better: Meta’s Prompt-Guard-86M matches the undefended agent (14.5% ASR), the rest leak 6.6 to 7.9%, and only deepset reaches 0%, at 31.2% utility. Tool-filtering also reaches 0%, again only by over-pruning

Table 4. Head-to-head on AgentDojo banking [7] ([†]detector added beyond AgentDojo’s shipped defences, $\uparrow/\downarrow$: higher/lower is better).

Defence (mechanism)	Benign util. (%) $\uparrow$	ASR (%) $\downarrow$
Undefended	50.0	11.8
Detector: Meta Prompt-Guard-86M [†]	37.5	14.5
Spotlighting (prompt delimiting)	43.8	10.5
Repeat-prompt (re-assert)	43.8	9.2
Detector: TestSavant-large [†]	43.8	7.9
Detector: ProtectAI DeBERTa	31.2	6.6
Detector: deepset DeBERTa [†]	31.2	0.0
Tool-filter (LLM tool-pruning)	25.0	0.0
DEON (ours, DPA guard)	37.5	0.0

tools (25% utility). DEON is the only defence that reaches **0%** attack success while keeping benign utility (37.5%), blocking calls in just the 2 of 16 benign tasks whose critical value comes from untrusted file content. That is the highest utility of any 0%-ASR defence. Because the DPA refuses by construction (Cor. 1(a)), this 0 is proposer-independent, which matters as undefended susceptibility is high and adversary-dependent (**11.8%** on the AgentDojo banking suite [7] here, up to 87–100% under our stronger synthetic ladder). App. C gives the per-task breakdown and how each baseline was built.

9.3 RQ3: Policy-inexpressible content egress

RQ3 tests the conformal layer where it matters: a *content-dependent egress* action the DPA *permits* but that leaks iff its free text is sensitive. The DPA inspects labels rather than free text. It is *blind* here (pass-through **1.00**) and a the specified logistic content detector of App. B supplies the signal. Over 30 seeds (Fig. 4b, Fig. 5a), exchangeable attacks are bounded at $0.089 \leq \alpha = 0.10$ at **100%** utility. The conformal layer carries this class, whereas an adaptive adversary obfuscating the content breaks exchangeability and drives pass-through to **0.328**, exactly the failure Cor. 1(b) predicts: the rewritten leaks score below τ and are admitted rather than abstained on.

A fixed cutoff ($\tau=0.5$) matches utility and on *this* detector even leaks less, yet cannot *track* a target, its pass-through swinging $0.158 \rightarrow 0.001$ as quality varies while the conformal guard holds $\approx \alpha$ (Fig. 5b). The distribution-free advantage is *target control* rather than lower per-pool leakage.

The experiment also quantifies one consequence of an A1 failure (§5). To the DPA, an action whose sensitive read is missing from its labels is indistinguishable from this content-dependent egress. It therefore falls outside the deterministic class. The remaining protection is $\leq \alpha$ under exchangeability and the measured pass-through is 0.328 after adaptive obfuscation.

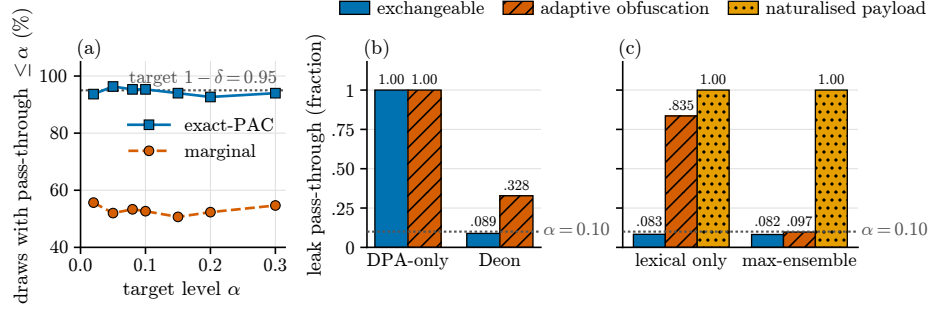


Fig. 4. Measured results. (a) RQ1: share of draws holding pass-through $\leq \alpha$. (b) RQ3: leak pass-through. (c) Hardening: lexical detector vs. three-detector ensemble. Dotted line in (b), (c): target $\alpha = 0.10$.

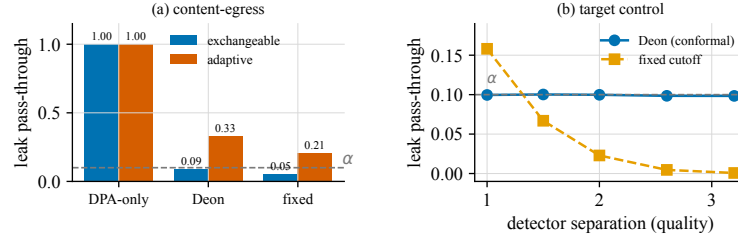


Fig. 5. RQ3 (30 seeds). (a) Content-egress pass-through at $\alpha=0.10$: the DPA is blind, conformal holds under exchangeability, and adaptive obfuscation defeats it. (b) Target control: conformal tracks $\alpha=0.10$ while a fixed cutoff swings.

9.4 RQ4: Hardening the conformal guard and cost

RQ3’s guard rests on *one* detector the adaptive adversary evades. We test two hardenings on the content-egress class ($\alpha=0.10$, $\delta=0.05$, 30 seeds, exact-PAC): an *ensemble* of separate detectors and *Mondrian* per-group calibration.

Ensembles resist single-detector evasion. We combine three *separate* detectors (lexical patterns, numeric digit-density, structural fragmentation) into one score by the *max* (admit only if low-risk to *every* detector). In Table 5 and Fig. 4c, exchangeable pass-through is $\leq \alpha$ for all four, but only lexical and the ensemble keep full utility. No *single* detector is both valid and useful. Only the ensemble is also robust to the lexical-targeting obfuscator. Under RQ3’s obfuscator aimed at lexical, its pass-through leaps to **0.835** while the ensemble holds **0.097** $\approx \alpha$ (an $8.6\times$ cut, also beating RQ3’s *blended* detector at 0.328): a max over *separate* conformal detectors is more evasion-robust than one blended score. It is not a cure, though: at the ensemble’s utility-preserving operating point an adversary that *naturalises* the payload (spelled digits, “alice at acme dot com”) slips past all three signals and drives it to **1.00** (structural alone blocks it only at its

Table 5. RQ4: leak pass-through $\mathbb{P}[\text{admit} \mid \text{leak}]$ for three separate detectors and their max-ensemble ($\alpha=0.10$, 30 seeds, CI half-widths ≤ 0.006).

Detector	Exchangeable		Adaptive-obf	Evade-all
	pass-through	utility	pass-through↓	pass-through↓
lexical	0.083	1.00	0.835	1.00
numeric	0.082	0.62	0.124	1.00
structural	0.088	0.09	0.037	0.068
ensemble (max)	0.082	1.00	0.097	1.00

unusable 0.09-utility threshold, Table 5). Hardening thus forces the attacker to defeat every signal at once but does not repeal Cor. 1(b).

Mondrian calibration restores control under shift. Exchangeability also breaks under benign *covariate shift*. Calibrating on a mix dominated by the easy score-regime group (0.6/0.3/0.1) and testing shifted toward the hard group (0.1/0.3/0.6), *marginal* calibration (one pooled τ) loses per-group control (overall 0.303, hard group **0.441**, $4.4 \times \alpha$), whereas *Mondrian* (τ per group) restores every group $\leq \alpha$ (worst **0.091**, overall 0.071) at a utility cost ($0.95 \rightarrow 0.71$), answering covariate shift given a group label, though not the *adaptive* shift above.

9.5 RQ5: Calibration validity, drift, and abstention recovery

We stress the conformal layer’s assumptions and the abstain valve ($\alpha=0.10$, $\delta=0.05$, exact-PAC, 200 to 400 draws or 30 to 40 seeds).

Label noise and small n_v (Fig. 6a,b). Under symmetric *calibration* label noise (fraction ρ flipped), PAC pass-through stays $\leq \alpha$ (0.085 at $\rho=0$, $\rightarrow 0$ by $\rho=0.05$) because mislabelled benign points only push τ down, at a utility cost (1.00, 0.77, 0.41 at $\rho=0, 0.05, 0.1$, and 0.17 by $\rho=0.3$): noise costs availability, never safety. Sweeping $n_v=20$ to 1280, the PAC rank k' climbs $0 \rightarrow 111$, while marginal holds the mean at ≈ 0.10 but per-draw only $\approx 50\%$.

Ties and detector miscalibration. With a *coarse* detector quantised to six levels, deterministic $\mathbf{s} \leq \tau$ over-admits ties and leaks **0.343** $\gg \alpha$ (the pathology Thm. 2 flags), while the executable augmented-score tie-break has observed mean **0.100** (rounded to three decimals), the naive strict-inequality repair ($\mathbf{s} < \tau$) over-corrects to 0.040. It is safe but wasteful. Under a *miscalibrated* detector (logit bias ± 0.15 , noise up to 0.30) the guard is invariant, holding 0.084 across the whole grid where a fixed cutoff swings 0.005 to 0.185, a $40\times$ spread. Only benign utility ($0.996 \rightarrow 0.88$) reflects the weaker detector.

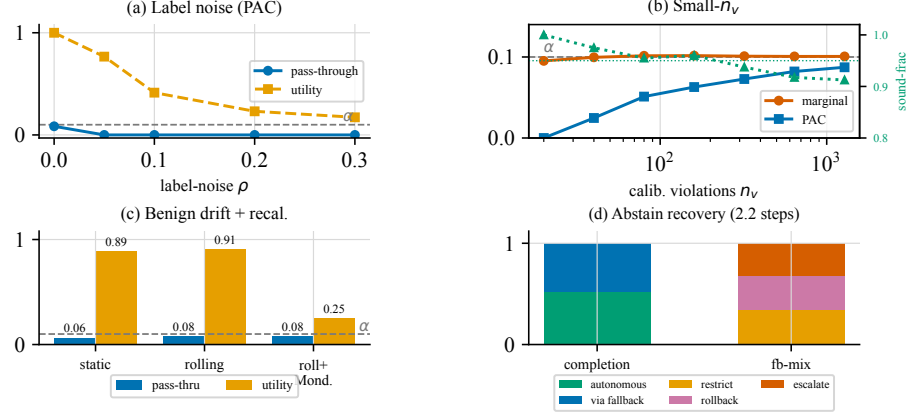


Fig. 6. RQ5 at $\alpha=0.10$. (a) Label noise trades utility for safety. (b) Small- n_v PAC pass-through and sound fraction. (c) Rolling/Mondrian recalibrate under drift. (d) Safe fallbacks restore completion without explicit violations (Monte Carlo model). Panels (a) to (c) are measured.

Benign drift and abstention recovery (Fig. 6c,d). On a 40-batch stream with drifting *benign* scores, a cheap drift monitor (benign-mean shift > 0.06) triggers rolling recalibration, lifting utility $0.89 \rightarrow 0.91$ at pass-through $\approx 0.08 \leq \alpha$ over 8.0 recalibrations at 0.04 ms each, while Mondrian keeps 0.076 at lower utility (0.25). In a Monte Carlo model of the safe-fallback ladder (restricted retry $\rightarrow$ rollback $\rightarrow$ escalation) with fixed branch probabilities, the raw abstain rate **0.225** still lifts completion $0.52 \rightarrow 1.00$ at **2.2** extra steps, no explicit violation executing (0.000) and admitted latent ones staying $0.095 \leq \alpha$: recovery restores liveness *without* weakening safety. The three fallback rungs fire in roughly equal proportion (0.34 restricted retry, 0.33 rollback/compensate, 0.32 human escalation). None of them is carrying the result alone.

Monitor cost. For the in-process Python automaton, monitoring time grows near-linearly in trace length ($R^2 = 0.99$ over lengths 2 to 12, slope $0.75 \mu\text{s}$ per action) at a few microseconds per trace (Prop. 2). This excludes detector inference, OPA, proxying, and network costs. Repetitions and standard deviations are recorded in the artifact.

9.6 Sensitivity of the conformal layer, and how close calibration is to deployment

Two related questions about the conformal layer are answered by the RQ5 stress tests only if those are read together: which parameters actually govern the calibration, and how much of the strong exchangeable result is owed to the calibration pool resembling the evaluated scenario. This subsection collects both

answers in one place. No new experiment is involved — every figure below is drawn from the measurements already reported — but the reading is different, and it is the reading an operator needs.

The parameters that govern τ , and what each one moves. Table 6 lists them. The pattern across the table is consistent and is the single most useful thing to know about the layer: in the tested calibration and noise settings, most perturbations trade utility rather than safety. Label noise drives pass-through to 0 and utility to 0.41. A poor detector holds pass-through at 0.098 and drops utility to 0.76. Miscalibration leaves pass-through at 0.084 and drops utility to 0.88. PAC conservatism costs 0.014 of pass-through relative to marginal and essentially no utility. The two exceptions are tied scores — where a deterministic $\leq$ comparison genuinely breaks the bound (0.343), which is why the randomised tie-break is the deployed default — and exchangeability, treated separately below.

Two parameters deserve a number rather than a direction. First, δ enters only through the rank: at $n_v \approx 1200$ and $\alpha = 0.10$ the marginal rank is $\lfloor \alpha(n_v + 1) \rfloor = 120$ while the exact binomial-tail rank at $\delta = 0.05$ is $k' = 103$, a 14% haircut that buys the move from “holds on $\approx 53\%$ of deployments” to “holds on 95% of deployments”. That is the whole price of the PAC variant, and it is the reason we treat PAC as the deployable mode rather than an aside. Second, n_v has a hard floor: at $n_v = 20$ the PAC rank is $k' = 0$, meaning no order statistic is certifiable and the guard abstains on every score. The rank becomes usable at $n_v = 40$ ($k' = 1$) and the PAC sound-fraction reaches 0.975. Practical guidance: a deployment needs on the order of 10^2 *labelled violations* rather than 10^2 labelled examples — at a 30% base rate that is a few hundred labelled actions.

How close the calibration pool is to the evaluated scenarios. In the exchangeable settings, the calibration and evaluation pools are drawn from the *same generator with the same parameters* and differ only in the random draw (§8.2). That is not an approximation of exchangeability. It is exchangeability, and this proximity is what drives the strong exchangeable result. A guard whose bound holds only under exchangeability, evaluated on exchangeable data, will hit its target. The RQ1 and RQ3 exchangeable columns therefore verify that the *implementation* attains the theoretical bound under its stated assumptions. They do not establish performance under deployment shift.

What makes the evaluation informative is the rest of it: we also measure the guard as the two distributions are pulled apart, along four independent axes, and report the degradation rather than only the best case. Table 7 is that ladder: row 1 is the exchangeable case behind the headline numbers, and rows 2–5 are the same guard under progressively less proximity. The bound survives identical distributions and a fixed score shift, degrades by $4.4\times$ under benign group-mixture shift, degrades by $3.7\times$ under content obfuscation, and loses its *confidence* (though not its mean) under intra-trace correlation. Only the first row is the fully exchangeable case, and it is the only row where DEON looks perfect. We therefore do not claim that the $\leq \alpha$ result transfers to a deployment whose traffic differs from its calibration pool. We claim that the failure is bounded in

Table 6. Sensitivity of the conformal layer ($\alpha=0.10$, $\delta=0.05$, exact-PAC unless noted). Every row but the last two trades utility rather than safety.

Parameter (range swept)	Pass-through	Utility	Reading
Target α (0.02–0.30)	tracks α : 0.020–0.299 marginal, 0.014–0.277 PAC	0.999–1.000	The dial works. PAC is uniformly conservative, by 0.006 at $\alpha=0.02$ to 0.022 at $\alpha=0.30$
Confidence δ (via rank, $n_v \approx 1200$)	marginal rank 120 $\rightarrow$ PAC rank 103	unchanged to 3 d.p.	14% rank haircut buys 53% $\rightarrow$ 95% of <i>deployments</i> sound
Sample size n_v (20–1280)	PAC 0.000 $\rightarrow$ 0.087; PAC rank k' : 0 $\rightarrow$ 111	0.000 at $k'=0$, else 0.996–1.000	$k'=0$ below $n_v \approx 40$: hard floor of $\sim 10^2$ labelled violations
Detector separation (1.0–3.2)	0.0996 $\rightarrow$ 0.0984 (flat)	0.76 $\rightarrow$ 1.00	Better detector $\rightarrow$ more utility, same risk (fixed cutoff: 0.158 $\rightarrow$ 0.001 risk instead)
Detector bias/noise (bias ± 0.15 , noise 0–0.30)	0.0844–0.0846 (flat)	0.996 $\rightarrow$ 0.88	Invariant; fixed cutoff swings 0.005–0.185 (40 $\times$)
Calibration label noise ρ (0–0.3)	0.085 $\rightarrow$ 0.000	1.00 $\rightarrow$ 0.17	Symmetric noise only pushes τ down: costs availability, never safety
Score resolution (6 quantisation levels)	0.343 with $\leq$; 0.040 with $<$; 0.100 randomised	—	Breaks the bound. Ties need the randomised break, which is the default
Calibration–deployment proximity	0.089 $\rightarrow$ 0.441 (see Table 7)	—	Breaks the bound. The only parameter that costs safety

the ways Table 7 shows, that three of the four shifts have a measured mitigation, and that adaptive obfuscation does not, which is why the latent-class bound is stated conditional on A4.

9.7 RQ6: Sequential dependence and scorer evolution

RQ1–RQ5 hold two assumptions of the conformal layer fixed: that per-step scores are exchangeable (they are *dependent* within one run), and that the scorer is fixed (it is *versioned* and retrains). RQ6 drops both, measured on the released prototype (Fig. 7, $\alpha=0.10$, $\delta=0.05$, exact-PAC, 400 draws or 40 seeds).

Inter-step dependence (Fig. 7a). We inject intra-trace AR(1) correlation of strength ρ across a violating trace’s steps, holding calibration *traces* fixed (80 calibration traces of 6 steps). PAC-calibrating a *step-level* τ treats dependent steps as exchangeable (the wrong unit, as the effective sample nears the trace count). The training-conditional guarantee erodes: the PAC sound-fraction falls $0.958 \rightarrow$ **0.777** as $\rho:0 \rightarrow 0.99$, below $1 - \delta=0.95$, while the i.i.d. baseline holds ≈ 0.96 (the mean pass-through stays near α . It is the *confidence* that breaks, which is exactly the failure a marginal-only report would hide). Reducing each

Table 7. Proximity ladder: the guard as the calibration pool and the evaluated traffic are pulled apart ($\alpha=0.10$).

Relation to calibration pool	Where measured	Pass-through	Mitigation measured here
1. Identical generator, independent draw (exchangeable)	RQ1 coverage; RQ3 exchangeable	$0.086\text{--}0.089 \leq \alpha$	— (this is the assumption holding)
2. Same generator, scorer version changed	RQ6 scorer evolution	$0.086 \leq \alpha$ with auto-recal; 0.026 if locked (stale, over-conservative)	Recalibrate on a <i>persistent</i> version change; hysteresis stops oscillation ($47 \rightarrow 0$ switches)
3. Same generator, benign group mixture reversed ($0.6/0.3/0.1 \rightarrow 0.1/0.3/0.6$)	RQ4 Mondrian	overall 0.303; worst group 0.441 ($4.4 \times \alpha$)	Mondrian per-group τ : worst group 0.091, at utility $0.95 \rightarrow 0.71$
4. Same generator, steps correlated within a trace (AR(1), $\rho \rightarrow 0.99$)	RQ6 dependence	mean stays $\approx \alpha$; PAC sound-fraction $0.958 \rightarrow$ 0.777	Calibrate per <i>trace</i> (max-risk step): 0.978 restored
5. Leaking content rewritten to evade the scorer (adaptive)	RQ3 / RQ4 adaptive	0.328 blended; 0.835 lexical; 0.097 max-ensemble; 1.00 evade-all	Ensembling helps ($8.6\times$) but does not repair it. Evading leaks are admitted rather than abstained on

trace to one exchangeable summary (its max-risk step) and PAC-calibrating at the *trace* level restores the bound (**0.978** at $\rho=0.99$). **Recommendation:** calibrate per trace rather than per step, under correlated scores.

Scorer evolution and adversarial oscillation (Fig. 7b). A versioned scorer drifts across releases: over 48 batches with four scorer versions, a *locked* profile grows stale (pass-through 0.026, safe but over-conservative), whereas *auto-recalibration* on a genuine version change restores control at **0.086** $\leq \alpha$ (3.0 recalibrations). An adversary who *toggles* the reported version every batch forces 47 profile switches under naive auto, but a *hysteresis* band (change must persist 3 batches) absorbs it at **0** switches and 0 spurious recalibrations. **Recommendation:** one profile per *signed* scorer version, recalibrated only on a persistent change.

10 Discussion, Limitations, and Threats to Validity

Scope of the guarantees. DEON depends on four conditions: correct and complete action labelling (A1), complete mediation (A2), an adequate policy (A3), and exchangeable calibration (A4). §5 identifies the responsible deployment component and failure mode for each. The deterministic results measure the guard under A1–A3 on a harness where these conditions hold by construction. They establish that our implementation realises the proved semantics and that the guarantee is uniform over proposers and adversaries. Establishing A1–A3 in production is a separate deployment responsibility.

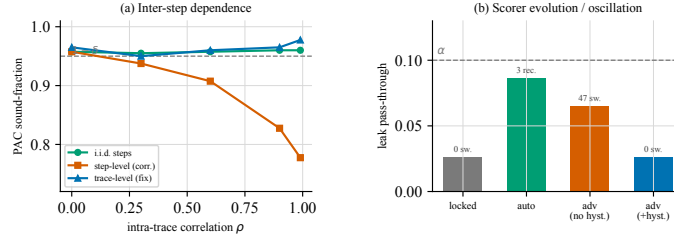


Fig. 7. RQ6 (measured). (a) Trace-level PAC restores soundness under inter-step dependence. (b) Hysteresis stops scorer-version oscillation while holding $\leq \alpha$.

Unevaluated deployment properties. Three deployment properties define the next validation boundary. (i) *Labelling error in practice.* The rate at which a real catalogue is mislabelled, and the distribution of over- versus under-labelling, needs a production service catalogue with an independently authored ground truth. DEON-BENCH supplies exact generator-side labels, while a production study requires independently authored ground truth. Section 9.3 instead isolates the cost of one under-labelled action. (ii) *Mediation in a distributed deployment.* Our interception point is a function boundary inside one process. The MCP proxy and gateway placements of §4 follow from the protocol structure and serve as design guidance rather than measured configurations. The reported results cover the in-process interception point rather than MCP-server deployment overhead or mediation completeness. The OPA-in-the-loop cost is likewise outside the measurements (App. E): the microsecond figures are the in-process automaton. (iii) *Policy authoring cost.* The evaluation contains one hand-authored Agent-Dojo policy over an independently designed catalogue. It demonstrates feasibility, while production-scale authoring effort and error rates require a broader catalogue study.

Operational limits. Exchangeability is the main statistical limit: strong shift raises true pass-through above α (the adaptive attack in Cor. 1(b)). Rolling recalibration, Mondrian groups, trace-level calibration, and PAC-conservatism cost utility rather than safety, but none repairs a scorer-evading adversary (§9.6, Table 7). Liveness is the second limit: forbidden actions never execute, but a weak proposer may still fail to finish. In a Monte Carlo model, a safe-fallback ladder lifts RQ5 completion $0.52 \rightarrow 1.00$ at 2.2 extra steps. The third is irreversibility: the guard cannot undo a side effect that already ran. Abstaining after a trigger has executed leaves a stranded obligation the monitor detects but cannot prevent (Thm. 1). Extending obligations to compensating-transaction frameworks and the guards to internal channels is future work.

Threats to validity. Construct. DEON-BENCH is synthetic and its ground truth is generator-side. This design makes exactness checkable and fixes the meaning of the 100% result: it certifies the monitor against the semantics rather than

against a human’s notion of a violation. *Internal.* Proposer and guard share no code, the monitor matches all 900 traces, and the calibration and evaluation pools are independent draws from the same generator. Section 9.6 then measures departures from that setting. *External.* DEON-BENCH and the AgentDojo head-to-head are research harnesses rather than production deployments [12,7]. The proposer ladder is six open-weights models served locally. Unguarded results may differ for frontier proposers. For policy-covered actions, the guarded result remains determined by the monitor semantics under A1–A3. The taint join is exact at field-tag granularity but not for implicit or timing channels, sub-field aliases, or cross-service identity. *Reproducibility.* The AgentDojo run uses a local non-deterministic proposer and is a single pass. Its per-condition percentages carry run-to-run variance we do not quantify. The artifact reproduces all other RQ1–RQ6 results bit-for-bit except machine-dependent timing.

11 Related Work

Runtime policy enforcement already provides many of the mechanisms that DEON uses. The relevant distinction is how those mechanisms behave when an agent influenced by untrusted content issues the request. Table 8 compares the design space along four central axes: whether the requester may be adversarial (*untrusted requester*), whether a decision can use session history, including counters and pending obligations (*trace state*), whether the decision propagates labels through an explicit-flow lattice (*lattice IFC*), and whether a distribution-free bound covers violations outside the policy language (*calibrated residual*). XACML “Obligations,” for example, are directives returned to the policy enforcement point, whereas the obligations here are liveness conditions over a trace. Likewise, **Indeterminate** denotes an evaluation error, whereas ABSTAIN is a calibrated response to residual risk. Section 11.1 develops these comparisons and motivates the DPA representation.

11.1 Runtime policy enforcement: XACML, usage control, security-by-contract

XACML and the attribute-based PDP. XACML [20] is the reference point for declarative runtime authorization: a policy decision point evaluates an XML-encoded request against attribute-based rules and returns **Permit**, **Deny**, **Not Applicable**, or **Indeterminate**, which a policy enforcement point applies. Role- and attribute-based access control [30,13] supplies the policy vocabulary, and the OPA/Rego stack [21] is the modern cloud-native realisation of the same architecture. Three properties of this model matter for an agentic composition.

First, the decision is *per request and stateless*. A PDP evaluates one `<Request>` against the attributes the PEP supplies. It has no memory of what this session already did. Everything in DEON’s state $q = (\sigma, u, \Omega, \lambda)$ — established propositions, per-resource counters, pending obligations, and the taint labelling — therefore has to live outside such a PDP, either in the PEP or

Table 8. DEON versus prior classes (◆ present, ◇ partial, blank absent, columns defined in the text).

Approach	Untrust. requester	Runtime per-action	Trace state	Lattice IFC	Hard guar.	Calib. residual	Abstain
LLM agents [36,31]		◇					
Prompt-inj. defences [7]	◆	◆					
XACML / ABAC		◆			◆		
PDP [20,30]							
Usage control /		◆	◆	◇	◆		
UCON [24,27]							
Security-by-contract [9,29]	◇	◆	◆		◆		
Policy engines		◆			◆		
(OPA/Rego) [21]							
Runtime verification [16,4]		◆	◆		◆		
SMT-backed	◇	◆	◆	◇	◆		
monitors [4,19]							
Lattice IFC [8]		◆	◆	◆	◆		
Conformal /		◆				◆	◆
selective [34,11]							
Static plan verifiers [6]			◆		◆		
DEON (ours)	◆	◆	◆	◆	◆	◆	◆

in application code, and its correctness is then nobody’s stated property. This is why the DPA’s *stateless fragment* maps cleanly onto Rego (one **deny** rule per clause, App. D) while Ω , **fin**, the quota counters, and the lattice join do not: they are trace properties, and a single-decision language cannot express them.

Second, XACML’s *Obligations* are not deontic obligations in the sense used here. An XACML obligation is a directive returned alongside a decision — “permit, and log this” — that the PEP must discharge at decision time. The obligations of Def. 2 are liveness properties of the trace: **charge** obliges a *later emitReceipt*, which may never be proposed, and whose absence is only detectable at end of trace. No single-decision evaluation can observe that, which is exactly what Thm. 1’s obligation clause formalises (and why the theorem is careful to say obligations are *detected* rather than enforced).

Third, the model assumes a *trustworthy requester*. The PEP is responsible for constructing a faithful request, and the attribute values it supplies are taken at face value. That assumption is sound when the PEP is application code and unsound when the requester is a language model steered by attacker-controlled text. DEON inherits the assumption only for the attributes it refuses to take from the proposer at all — which is precisely what assumption A1 (§5) makes explicit, and the reason the labelling must be authored by the catalogue owner rather than derived from the request.

Usage control. UCON_{ABC} [24] is the closest classical relative, and it is closer than the runtime-verification line. It goes beyond access control precisely on the two axes we need: *mutability* of attributes (a decision updates the state it decided on) and *continuity* of enforcement (pre-, ongoing-, and post-decisions rather than a single gate), with authorizations, obligations, and conditions as first-class elements. Distributed usage control [27] extends this to data that has left the originating domain, with data-flow tracking as part of the enforcement

story. Read against Table 8, the DPA’s mutable counters, its obligation lifecycle, and its ongoing per-action gate are established usage-control concepts rather than contributions of this paper.

The DPA specializes usage control for this setting in three ways. (i) It is a *concrete finite automaton with an accepting condition*: δ is a total function on (q, a) whose every clause is a bounded lookup, fin is the end-of-trace obligation check, and together they give a per-trace certificate and a stated $O(kf)$ cost (Prop. 2). This construction supports per-action monitoring at $0.75 \mu\text{s}$ per action (§9.5) and ties the soundness statement to the executable monitor. (ii) The lattice join is a *transition effect* rather than a separate subsystem: propagating $\lambda'(w) = \lambda_0(w) \sqcup \bigsqcup_{r \in R(a)} \lambda(r)$ inside δ is what catches laundering (§3) without bolting an information-flow tracker onto a usage-control engine, and it is why *laundering* is one of the six classes RQ1 measures at 100% recall. (iii) Most importantly, usage control assumes the subject’s attributes are supplied truthfully. An agentic composition breaks that, and the DPA is designed so that *nothing the requester produces* beyond the operation name and arguments enters the decision.

Security-by-contract, and Rasori et al. Security-by-contract [9] pairs mobile code with a contract describing its security-relevant behaviour, matches that contract against the platform’s policy before execution, and falls back to runtime monitoring when the match fails. Rasori et al. [29] give the most recent realisation of this idea in a usage control setting: they implement and enforce security-by-contract on top of a UCON/XACML-based usage-control architecture, mapping contracts and policies into usage-control policies for smart-home applications, and evaluate the resulting enforcement overhead. Structurally, this is the same two-mode design as DEON: establish what can be established ahead of time, and monitor the rest at run time.

The difference is what plays the role of the contract. In $S \times C$ the untrusted artifact is *code*, which is fixed, inspectable, and signable. The contract can be shipped with it and verified or trusted through a signature — and a successful match lets the platform skip monitoring entirely. An LLM proposer ships no contract: its behaviour is not fixed (it is a function of attacker-controlled input at each step), it cannot be verified against a claimed contract, and a signature over its weights says nothing about the actions it will propose next. An agent cannot supply a contract that justifies disabling monitoring. Monitoring remains active. The useful analogue of a contract instead resides in the platform: it is the typed catalogue label of assumption A1, authored by whoever admits a service, describing what an *operation* does rather than what the *agent* intends. Thus A1 relocates the trusted description from the code supplier to the catalogue owner. Rasori et al.’s enforcement architecture is also a useful template for A2: their contribution includes where the enforcement point sits and what it costs, which is the question §4 has to answer for MCP.

Residual-risk calibration. XACML, usage control, security-by-contract, runtime verification, and information-flow control decide over the rules and attributes

presented to them. Their default behavior is configuration-specific. A content-dependent danger outside that representation remains a residual for the configured policy (§9.3). DEON adds to this interface a finite-sample admission bound under exchangeability and a first-class ABSTAIN verdict. Conversely, the cited conformal and selective-prediction literature [34,1,2,3,5,11,33] supplies that calibration machinery but does not itself encode the service policy and executed-prefix state used here. DEON combines the two approaches: the policy layer governs expressible violations, while the statistical layer calibrates the residual and abstains on actions above its threshold.

11.2 Agentic systems, attacks, and adjacent defences

Agentic service composition. Recent service-computing work builds compositions *with* LLMs: *Compositio Prompto* places them inside an automated service-computing architecture [26], NL2TOSCA compiles requirements into TOSCA topologies [17], MCPybarra generates MCP services [25], and agentic loops [31] do the same for a single agent. Each produces a plan that is subsequently executed. DEON inserts an independent decision point at that boundary. Any of these planners can serve as the proposer in Fig. 2. Trace-based audits estimate how reliably an agent completes its tasks after the run [32]. DEON acts before each step and stops the action. Trust also enters service *selection* statistically, through QoS and trust prediction [14], but that predicts which service to pick rather than what may execute now, and binds no adversary.

LLM agents and tool use. Tool-augmented models [28] and agentic loops [36,31] improve proposals but still treat the model output as the plan to run. DEON instead governs *execution*. An admitted action never depends on trusting the proposer.

Agent security and prompt injection. Indirect prompt injection compromises LLM applications through consumed content [12], AgentDojo [7] quantifies how far an agent can be subverted, and policy violation remains a top LLM risk [23]. These works establish the threat. DEON adds an A1–A3-conditional guarantee for the action-level safety fragment and an A4-conditional per-action admission bound for the calibrated residual.

Information flow and runtime verification. Lattice information flow [8] gives the taint model the DPA embeds. Runtime verification [16,4] gives the monitoring model, and SMT-grounded monitors [19,4] are more expressive than the DPA for cross-call value constraints and complex temporal patterns, at higher cost and with no calibrated residual layer. DEON is orthogonal to that expressiveness axis: a richer symbolic monitor could replace the DPA beneath the same conformal guard and ABSTAIN (App. F). Unlike static plan verifiers [6], DEON certifies each action at run time with no proposer model.

Runtime middleware for agents. ClawGuard [37] is the closest middleware analogue, enforcing induced rules at the tool boundary against indirect injection and overlapping the DPA’s pre-invocation blocking. The residual bound, calibrated abstain, and obligation lifecycle remain DEON’s. The AI Workflow Store [10] hardens reusable workflows, after which DEON adds runtime governance. To our knowledge DEON is the first system to combine a deontic trace monitor, a distribution-free residual bound, and a first-class abstain for agentic service composition under attack.

12 Conclusion

An agent’s proposed action carries no execution authority by itself. We make that distinction executable by placing a two-layer guard between the proposer and service execution. Our deontic policy automaton (DPA) enforces the action-level safety fragment and detects terminally undischarged obligations, while the conformal layer calibrates admission for violations outside that policy language. On DEON-BENCH, the monitor matches all 900 labelled traces, the guard meets its target at $\approx 100\%$ utility and is PAC-valid on 93–96% of draws per level, and content-egress leakage falls to 0.089 from a blind baseline of 1.0. Across the proposer ladder, the DPA reduces attack success from 87–100% to **0%** with microsecond-scale monitoring overhead.

These results define a precise deployment contract: every action reaches the guard with correct labels, no effectful action bypasses the interception point, and the policy represents the operator’s intent (A1–A3, §5). Current service schemas do not provide all labels required by A1, and those labels cannot be accepted from the untrusted proposer. A deployment must therefore author and validate them separately. An under-labelled action is outside the deterministic guarantee and is assessed only by the statistical layer. §9.3 measures pass-through of 0.089 under exchangeability and 0.328 under adaptation for this case. This boundary points directly to the next step: derive and sign catalogue labels for MCP servers, then measure their error rate on production catalogues.

Acknowledgments. We acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for supporting this study.

References

1. Angelopoulos, A.N., Bates, S.: Conformal prediction: A gentle introduction. *Found. Trends Mach. Learn.* **16**(4), 494–591 (2023)
2. Angelopoulos, A.N., Bates, S., Candès, E.J., Jordan, M.I., Lei, L.: Learn then test: Calibrating predictive algorithms to achieve risk control. *Ann. Appl. Stat.* **19**(2), 1641–1662 (2025). <https://doi.org/10.1214/24-A0AS1998>
3. Angelopoulos, A.N., Bates, S., Fisch, A., Lei, L., Schuster, T.: Conformal risk control. In: *ICLR* (2024)
4. Bauer, A., Leucker, M., Schallhart, C.: Runtime verification for LTL and TLTL. *ACM Trans. Softw. Eng. Methodol.* **20**(4), 14:1–14:64 (2011)

5. Chow, C.K.: On optimum recognition error and reject tradeoff. *IEEE Trans. Inf. Theory* **16**(1), 41–46 (1970)
6. De Giacomo, G., Vardi, M.Y.: Linear temporal logic and linear dynamic logic on finite traces. In: *IJCAI*. pp. 854–860 (2013)
7. Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F.: AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: *NeurIPS* (2024)
8. Denning, D.E.: A lattice model of secure information flow. *Commun. ACM* **19**(5), 236–243 (1976)
9. Dragoni, N., Massacci, F., Naliuka, K., Siahaan, I.: Security-by-contract: Toward a semantics for digital signatures on mobile code. In: *EuroPKI. LNCS*, vol. 4582, pp. 297–312. Springer (2007). https://doi.org/10.1007/978-3-540-73408-6_21
10. Geambasu, R., Raykova, M., Tholoniati, P., Tiwari, T., Tsai, L., Zhang, W.: Engineering robustness into personal agents with the AI workflow store. *arXiv:2605.10907* (2026)
11. Geifman, Y., El-Yaniv, R.: Selective classification for deep neural networks. In: *NeurIPS* (2017)
12. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: *AISec* (2023)
13. Hu, V.C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., Scarfone, K.: Guide to attribute based access control (ABAC) definition and considerations. Tech. Rep. SP 800-162, National Institute of Standards and Technology (2014). <https://doi.org/10.6028/NIST.SP.800-162>
14. Kumar, S., Chattopadhyay, S.: TRQP: Trust-aware real-time QoS prediction framework using graph-based learning. In: *ICSOC*. pp. 143–152. Springer (2022)
15. Le, T.H.C., Tran, P.H.V., Mai Duc, T., Le, X.B.: LTM-RAG: Longitudinal trust mining for poison-resilient retrieval-augmented generation. In: *Proc. IEEE ICDM* (2026), to appear
16. Leucker, M., Schallhart, C.: A brief account of runtime verification. *J. Log. Algebr. Program.* **78**(5), 293–303 (2009)
17. Meflah, W., Brabra, H., Acheli, M., Ben Douma, H., Sellami, M., Gaaloul, W., Zeghlache, D.: From natural language to TOSCA: Leveraging LLMs for automated service composition. In: *ICSOC*. pp. 3–18. Springer (2026). https://doi.org/10.1007/978-981-95-5012-8_1
18. Model Context Protocol Project: Model context protocol specification, revision 2025-03-26 (2025), tool-annotation revision. Accessed 24 September 2026. <https://modelcontextprotocol.io/specification/2025-03-26/changelog>
19. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: *TACAS*. pp. 337–340 (2008)
20. OASIS: eXtensible access control markup language (XACML) version 3.0 (2013), oASIS Standard, 22 January 2013. <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>
21. Open Policy Agent Project: Open policy agent documentation (2026), accessed 24 September 2026. <https://www.openpolicyagent.org/docs/>
22. OpenAPI Initiative: OpenAPI specification v3.1.0 (2021), 15 February 2021. <https://spec.openapis.org/oas/v3.1.0.html>
23. OWASP Foundation: OWASP top 10 for large language model applications (2023), <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

24. Park, J., Sandhu, R.: The $UCON_{ABC}$ usage control model. *ACM Trans. Inf. Syst. Secur.* **7**(1), 128–174 (2004). <https://doi.org/10.1145/984334.984339>
25. Peng, B., Liu, M., Liu, Y., Tian, C., Yu, S., Wang, Z.: MCPybarra: A multi-agent framework for low-cost, high-quality MCP service generation. In: *ICSOC*. pp. 51–65. Springer (2026). https://doi.org/10.1007/978-981-95-5012-8_4
26. Pesl, R.D., Mombrey, C., Klein, K., Zyberaj, D., Georgievski, I., Becker, S., Herzwurm, G., Aiello, M.: Compositio prompto: An architecture to employ large language models in automated service computing. In: *ICSOC*. pp. 276–286. Springer (2024). https://doi.org/10.1007/978-981-96-0808-9_20
27. Pretschner, A., Hilty, M., Basin, D.: Distributed usage control. *Commun. ACM* **49**(9), 39–44 (2006). <https://doi.org/10.1145/1151030.1151053>
28. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., Sun, M.: ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In: *ICLR* (2024)
29. Rasori, M., Mori, P., Saracino, A., Aldini, A.: Exploiting usage control for implementation and enforcement of security by contract. *Internet of Things* **33**, 101697 (2025). <https://doi.org/10.1016/j.iot.2025.101697>
30. Sandhu, R.S., Coyne, E.J., Feinstein, H.L., Youman, C.E.: Role-based access control models. *IEEE Comput.* **29**(2), 38–47 (1996)
31. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: *NeurIPS* (2023)
32. Tran-Truong, P.T., Le, X.B.: TraceToChain: Auditing LLM-agent reliability with absorbing Markov chains. In: *Proc. IEEE ISSRE* (2026), to appear
33. Vovk, V.: Conditional validity of inductive conformal predictors. *Mach. Learn.* **92**(2–3), 349–376 (2013)
34. Vovk, V., Gammerman, A., Shafer, G.: *Algorithmic Learning in a Random World*. Springer (2005)
35. von Wright, G.H.: Deontic logic. *Mind* **60**(237), 1–15 (1951)
36. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: *ICLR* (2023)
37. Zhao, W., Li, Z., Zhang, P., Sun, J.: ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection. *arXiv:2604.11790* (2026)

A Proofs

This appendix gives the full proofs of the results stated in §7. Notation is as defined there. Throughout, A1–A3 of §5 are in force for Thm. 1, Cor. 1(a), and Prop. 1’s blocking clause. A4 is additionally in force for Thm. 2 and Cor. 1(b), in its i.i.d. (PAC) form wherever the exact-PAC part is used.

Theorem 1 (DPA soundness)

Proof. Write $\pi = a_1 \cdots a_k$ for the executed trace and q_i for the monitor state after a_i , with q_0 the initial state.

(*State adequacy.*) We first show by induction on i that q_i is exactly the deontic effect of the prefix $a_1 \cdots a_i$, i.e. σ_i is the set of propositions granted by

that prefix, u_i its per-resource usage counts, Ω_i its triggered-but-undischarged obligations, and λ_i the taint labelling induced by its reads and writes. The base case $i = 0$ holds by definition of q_0 . For the step, assume it for $i - 1$. Because A2 holds, a_i reached the guard before it executed, and because it executed it must have had $\delta(q_{i-1}, a_i) = (ok, q_i)$ with ok true (Algorithm 1, line 7). By Def. 3, q_i is obtained from q_{i-1} by adding a_i 's granted propositions, incrementing its resource counter, discharging from Ω_{i-1} the obligation satisfied by a_i , registering $O(a_i)$ for the next prefix, and applying the lattice join to a_i 's write set. Under A1 those four pieces of information (a_i 's grants, its meter, its obligation trigger, its read/write field tags) are the true ones for the underlying service invocation. Each update matches the real effect, and the induction hypothesis gives the rest.

(*Safety.*) Let a_i be any executed action. As above, ok held at q_{i-1} . By the four clauses of Def. 3: (i) a_i is not in the forbidden-operation set of Π , (ii) $P(a_i) \subseteq \sigma_{i-1}$, and by state adequacy σ_{i-1} is exactly the prefix's established propositions. No permission precondition is violated, (iii) every field in $R(a_i)$ flows to the clearance of a_i 's sink, and since λ_{i-1} carries joined taint, this covers laundered flows as well as direct ones at field-tag granularity, (iv) u_{i-1} plus a_i 's charge is within Π 's quota. These are precisely the four safety clauses of Def. 2. π violates none of them. Refusal is what enforces this: an action failing any clause is returned as BLOCK and never executed, and q is left unchanged. A blocked action cannot perturb the state either. Because δ is a function of (q, Π) alone and reads nothing the proposer emits beyond a_i 's identity and arguments, the argument is uniform over proposers and adversaries.

(*Obligations.*) An obligation is the liveness property “if the trigger occurs, the obliged action occurs later”. Let a_i trigger $o = O(a_i)$. At the moment a_i is adjudicated, whether o will later be proposed is not determined by (q_{i-1}, a_i, Π) : it depends on the proposer's future output, which the monitor may not read, and on the remaining step budget. A blocking monitor therefore has exactly two options, and neither enforces the property. It may refuse every trigger, which enforces the obligation vacuously and makes the policy useless, or it may admit the trigger, in which case o is registered in Ω and $\text{fin}(q_k)$ reports at end of trace whether $\Omega_k = \emptyset$. DEON takes the second option. Since a_i has already executed and its side effect cannot be revoked by the guard, a trace that subsequently ABSTAINS (or exhausts its budget) with $o \in \Omega$ ends in a detected but unprevented violation. Hence obligations are *detected* rather than *enforced*, which is the theorem's second clause. $\square$

Remark (why A1–A3 are per-action switches). The induction above uses A1 once per action, at the step where it identifies a_i 's declared tags with the invocation's real effect. If A1 fails for a single action a_j — say its read set omits a `pii` field — the induction breaks only at j , but it breaks silently: clause (iii) evaluates over an incomplete $R(a_j)$ and returns *ok*. No refusal is emitted, and λ_j thereafter under-approximates the true taint, which can propagate the error forward. Likewise A2 is used once per action, at the point where “executed” is taken to imply “adjudicated”. An unmediated action falsifies that implication for itself and contributes no constraint at all. A3 does not enter the proof, because the

theorem is a statement *about* Π : it is what makes the conclusion “no violation of Π ” rather than “no harmful action”.

Theorem 2 (conformal admission bound)

Proof. (i) *Marginal.* Let S be the nonconformity score of a fresh violation and $V_1, \dots, V_{n_v}$ the calibration violation scores, and assume the $n_v + 1$ scores are exchangeable (A4). Exchangeability makes the rank of S among the $n_v + 1$ values uniform on $\{1, \dots, n_v + 1\}$ when there are no ties. The guard admits iff $S \leq \tau = v_{(k)}$, which happens iff S ’s rank is at most k , an event of probability $k/(n_v + 1)$. With $k = \lfloor \alpha(n_v + 1) \rfloor$ this is $\lfloor \alpha(n_v + 1) \rfloor / (n_v + 1) \leq \alpha$, giving the stated chain. With ties the rank is no longer uniform and the deterministic comparison $S \leq \tau$ can admit an entire tied block, which is the pathology measured at 0.343 in §9.5. Ordering independent augmented pairs (S, U) with $U \sim \text{Unif}(0, 1)$ restores uniformity of the randomized rank and hence the bound. Note what is and is not bounded: the statement conditions on the event “the action is a violation”. It bounds pass-through $\mathbb{P}[\text{admit} \mid \text{violation}]$, and therefore also the joint $\mathbb{P}[\text{admit} \wedge \text{violation}] \leq \alpha$. It says nothing about the reverse conditional $\mathbb{P}[\text{violation} \mid \text{admit}]$, which depends on the base rate.

(ii) *PAC (exact).* Here the hypothesis is the i.i.d. form of A4: $V_1, \dots, V_{n_v}$ are i.i.d. with continuous CDF F , the law of a deployment violation score, and the calibration sample is independent of the deployment violations. This is strictly stronger than the exchangeability used in (i): exchangeability supports the marginal rank argument, but a training-conditional statement needs a fixed F from which the sample is drawn (an exchangeable mixture of i.i.d. laws satisfies (i) yet can violate (ii) conditional on the mixing component). Under this hypothesis the probability-integral transform makes $F(V_1), \dots, F(V_{n_v})$ i.i.d. uniform on $[0, 1]$ and, F being non-decreasing, $F(V_{(j)})$ is their j -th order statistic, hence $F(V_{(k')}) \sim \text{Beta}(k', n_v + 1 - k')$. The Beta–Binomial identity $\mathbb{P}[\text{Beta}(k', n_v + 1 - k') \leq \alpha] = \mathbb{P}[\text{Bin}(n_v, \alpha) \geq k']$ turns the training-conditional coverage event into a binomial tail. Choosing k' as the largest rank with $\mathbb{P}[\text{Bin}(n_v, \alpha) \geq k'] \geq 1 - \delta$ therefore yields $\mathbb{P}_{\text{cal}}[F(\tau) \leq \alpha] \geq 1 - \delta$ with $\tau = v_{(k')}$, i.e. with probability at least $1 - \delta$ over the draw of the calibration set, the *realised* pass-through of the deployed threshold is at most α . The construction is exact rather than asymptotic and uses no property of F beyond continuity. It is distribution-free [33]. Because $\lfloor \alpha(n_v + 1) \rfloor$ is a mode of $\text{Bin}(n_v, \alpha)$ and the binomial mass at or below it exceeds $1/4$, we have $\mathbb{P}[\text{Bin}(n_v, \alpha) \geq \lfloor \alpha(n_v + 1) \rfloor + 1] < 3/4$. $k' \leq \lfloor \alpha(n_v + 1) \rfloor$ for every $\delta \leq 1/4$, including the $\delta = 0.05$ used throughout (for δ near $1/2$ it can fail). The PAC threshold is then never more permissive than the marginal one, which is the conservatism measured in §9.1, and $k' = 0$ whenever n_v is too small for any rank to satisfy the tail condition, which is the hard floor measured at $n_v = 20$ in §9.6. $\square$

Corollary 1 (attack-success bounds)

Proof. (a) Let A be any adversary and P any proposer, and let a be an action that would constitute a DPA-covered violation (prohibited operation, unmet precondition, direct or laundered flow at field-tag granularity, or quota excess) in the state reached so far. Under A1–A3, Thm. 1(safety) applies to every executed action. a cannot be among them. Equivalently, $\delta_\Pi(q, a)$ returns $\neg ok$ and Algorithm 1 line 7 refuses it. Neither A nor P appears in this argument: δ_Π is a function of (q, Π) and a ’s declared identity and arguments only. Hence the execution probability is exactly 0, uniformly over (A, P) . The quantifier is over adversaries and proposers rather than over deployments — A1–A3 are the deployment-side hypotheses, and Thm. 1’s remark shows each failure removes specific actions from the scope rather than degrading the bound continuously.

(b) A latent violation is invisible to δ_Π by definition. It executes iff the conformal guard admits it, i.e. iff its score is $\leq \tau$. If its score is exchangeable with the calibration violation scores, Thm. 2(i) bounds that probability by α (or, in the PAC variant and under the i.i.d. form of A4, bounds the realised rate by α with probability $1 - \delta$). The hypothesis is what the adversary attacks: the score is computed on adversary-supplied content. An adversary who shifts the latent-score law — by rewriting a payload until it scores low — makes the fresh score non-exchangeable with calibration, and nothing in the argument survives. The bound is therefore *not* uniform over adversaries, and in the limit of a perfectly evading adversary the pass-through approaches 1, which is the 0.328 (blended detector) and 1.00 (evade-all against the ensemble) measured in §9.3 and §9.4. $\square$

Proposition 1 (benign utility)

Proof. A benign action satisfies Π . δ_Π returns ok and it is never BLOCKED. By Def. 4 it is then ADMITTED iff $s \leq \tau$. The benign admission rate is $\mathbb{P}[s \leq \tau \mid y=0]$, which is the claim’s first part. For optimality, note that $\mathbb{P}[s \leq \tau \mid y=0]$ is non-decreasing in τ , as is the pass-through $\mathbb{P}[s \leq \tau \mid y=1]$. Restrict attention to thresholds that are certifiably risk-controlled, i.e. calibration order statistics $v_{(j)}$ with $j \leq \lfloor \alpha(n_v + 1) \rfloor$ (any larger rank has a marginal bound exceeding α by Thm. 2(i)). Since $v_{(1)} \leq \dots \leq v_{(k)}$, the largest admissible rank $k = \lfloor \alpha(n_v + 1) \rfloor$ gives the largest such τ , hence the largest benign admission rate. No risk-controlled rule in this family admits more. $\square$

The same monotonicity explains the shape of every sensitivity row in Table 6: a perturbation that pushes τ down (label noise, PAC conservatism, a small n_v) reduces both pass-through and utility. It can only cost availability. A perturbation that changes the violation-score law without changing τ (adaptive obfuscation, covariate shift) moves pass-through without moving utility, which is why those are the only rows that cost safety.

Proposition 2 (efficient monitoring)

Proof. Let f bound the number of fields, propositions, and quota entries a single action touches. Evaluating $\delta(q, a)$ requires: one membership test of a in the forbidden set ($O(1)$ hashed), one subset test $P(a) \subseteq \sigma(O(f))$, the flow check, which compares each $f' \in R(a)$ against the sink clearance ($O(f)$ lattice comparisons, each $O(1)$ in a fixed finite lattice), and one counter comparison ($O(1)$). The state update adds the lattice join over $W(a)$ ($O(f)$), one counter increment, one obligation registration, and one obligation discharge (each $O(1)$ hashed). Hence δ is $O(f)$ and a length- k trace is $O(kf)$. fin is a single emptiness test. Calibration sorts the $n_v \leq n$ violation scores once, $O(n \log n)$, and is performed offline. Per-step online cost is thus one $O(f)$ transition plus one invocation of the scorer, whose cost is the detector’s and is independent of the guard. $\square$

B Content-egress detector and calibration data

This section expands the RQ3/RQ4 content-egress setup for reproducibility. The released content-sensitivity detector (`content.py`) is a logistic over a sparse pattern signal plus continuous text features. For text t it forms

$$z(t) = 1.5m(t) + 6.0d(t) + 1.3a(t) + 2.5\ell(t) - 2.2, \quad s(t) = \sigma(z(t)),$$

where m is a weighted marker mass, d is digit density, a is the @ count, and ℓ is the fraction of long tokens. The markers are e-mail, card-number, national-identifier, and secret-key patterns (weight 1.0), sensitivity words such as *confidential*, *PII*, *credentials*, and *password* (weight 0.6), and weak lexical cues such as *customer* and *account* (weight 0.25). The logistic output is continuous in its features, but leak texts are composed from finite marker banks. Scores do repeat: at $n_{\text{cal}}=3000$ only 31–36% of calibration violation scores are distinct, and 3 to 21 of them tie at the PAC threshold across the 30 seeds. RQ3 uses the deterministic comparison $s \leq \tau$. For this detector the untied condition of Thm. 2 holds only approximately and the measured rates (0.089 exchangeable) are the evidence that the target is met. The detector is deliberately imperfect. It misses obfuscated markers, which is the adaptive-evasion failure mode we study, and it fires mildly on benign mentions of *customer* or *account*.

Calibration and evaluation pools (`build_content_pool`) draw benign texts and, with leak rate 0.30, leaking texts that carry real sensitive fields across three leakage classes: PII (e-mail, card, national identifier), secrets (API keys and tokens), and sensitivity-labelled prose. Ground-truth labels are generator-side. $y=1$ holds exactly when the text truly carries sensitive data. Manual annotation is therefore unnecessary and the labelling protocol is exact by construction. The *adaptive* pool obfuscates leaking content with character insertions and homograph substitutions that break the pattern markers while preserving meaning, which breaks exchangeability with a plaintext-calibrated threshold. A stronger detector swaps in without recalibration because the guard calibrates on whatever score distribution the detector emits. For RQ4 the three ensemble detectors

key on genuinely different signals (lexical pattern coverage, numeric digit density, and structural fragmentation). An adversary tuned to evade one signal still faces the other two under the max-ensemble.

C AgentDojo protocol and per-task breakdown

All nine conditions in the AgentDojo head-to-head run under one configuration and differ only in the defence wrapper. They share the same `qwen2.5:7b` proposer (served through Ollama’s OpenAI-compatible endpoint at fixed decoding settings), the same banking tool catalogue, the same 16 user tasks crossed with the five injection tasks $\{0, 3, 4, 5, 7\}$, the same *important-instructions* attack, and the same five-iteration budget (`max_iters=5`). The proposer is a local Ollama model. The run is a single non-deterministic pass rather than a fixed-seed one. The 76 non-degenerate combinations exclude the four pairs in which the user’s own authenticated instruction already names the injection’s target sink, where no runtime monitor should intervene.

Baseline construction and settings. We build each competing defence exactly as AgentDojo constructs it in `AgentPipeline.from_config`. The comparison is faithful to the upstream implementations, and all wrappers live in `agentdojo_deon.py` (the `build_pipeline` function and the `_DETECTORS` registry), reproduced by `run_agentdojo.sh`. *Undefended* is the plain proposer-and-tools loop. *Spotlighting* (spotlighting-with-delimiting) appends to the system prompt an instruction to distrust anything between « and » and wraps every tool output in those delimiters. *Repeat-prompt* re-injects the authenticated user instruction after each tool result. *Tool-filter* runs an LLM pre-pass (AgentDojo’s `OpenAILLMToolFilter`) that prunes the available tools to those the task needs before the proposer runs. The three *ML detectors* wrap AgentDojo’s `TransformersBasedPIDetector` (threshold 0.5, message mode) around every tool output and flag an injection when the model’s non-safe label scores above 0.5: ProtectAI DeBERTa-v3 (`protectai/deberta-v3-base-prompt-injection-v2`, safe label `SAFE`), deepset DeBERTa-v3 (`deepset/deberta-v3-base-injection`, safe label `LEGIT`), and TestSavant’s large defender (`testsavantai/prompt-injection-defender-large-v0`, safe label `SAFE`), and Meta Prompt-Guard-86M (safe label `BENIGN` over its `BENIGN/INJECTION/JAILBREAK` head). Prompt-Guard’s weights are openly (Llama) licensed but sit behind a manual license-acceptance gate on the `meta-llama` repo. We load a public re-upload whose DeBERTa-v2 architecture, `BENIGN/INJECTION/JAILBREAK` label set, and 278.8M parameters match the original. The 2025 Prompt-Guard-2 ships only as gated or ONNX weights and is omitted. DEON is the DPA counterparty and value-provenance guard of the main paper, admitting a money or credential action only when its recipient or new value is authorised by the user’s instruction or an established counterparty. A detector or tool-filter flag, like a DPA refusal, converts the flagged tool call into a blocked or abstained step.

Table 9 gives the per-task benign-completion breakdown for the undefended run and DEON. DEON blocks calls in exactly two benign tasks, `user_task_0`

Table 9. AgentDojo banking per-task benign completion (qwen2.5:7b), undefended vs. DEON.

Task	Undef.	DEON	Task	Undef.	DEON
user_task_0	✓	—	user_task_8	✓	✓
user_task_1	—	—	user_task_9	✓	✓
user_task_2	—	—	user_task_10	✓	✓
user_task_3	—	—	user_task_11	—	—
user_task_4	✓	—	user_task_12	—	—
user_task_5	✓	✓	user_task_13	—	—
user_task_6	—	—	user_task_14	✓	✓
user_task_7	✓	✓	user_task_15	—	—

and `user_task_4`, whose critical value (a new payee or credential) comes from untrusted file content that the user’s instruction never named. On every other task DEON matches undefended completion. Under injection DEON admits zero attacker actions at every combination, because no injected instruction can introduce a fresh authorised sink. This is the by-construction refusal of Cor. 1(a).

D A representative Rego/OPA policy

The benchmark’s deontic policy (`catalog.py`) has a stateless fragment that maps directly to Rego, one `deny` rule per hard gate over the state document the automaton’s transition supplies. Listing 1.1 illustrates the mapping for the payment catalogue, covering prohibitions, explicit or laundered flow at field-tag granularity, required preconditions, and a quota. We wrote the listing by hand to show the correspondence. The obligation `charge⇒emitReceipt` is a trace-level liveness property that the automaton finalises at end-of-trace, because a single Rego evaluation decides one admission and cannot observe later discharge.

Listing 1.1. Representative Rego fragment for the payment policy. `input` is the state document the DPA transition supplies for the action under review.

```
package deon.banking
default allow = false
allow { count(deny) == 0 }

# prohibited actions
prohibited := {"exfiltrate", "dumpDB"}
deny["prohibited action"] { prohibited[input.action] }

# explicit/laundered flow: no pii/internal field into a public sink
deny["disallowed flow"] {
  input.field_class[input.args.field] == "pii"
  input.sink_class == "public"
}

# required preconditions (charge/refund: authenticated; ship: paid)
require_before := {"charge": {"authenticated"},
  "refund": {"authenticated"}, "ship": {"paid"}}
deny[sprintf("missing precondition %v", [need])] {
  need := require_before[input.action][_]
}
```

```

    not input.state[need]
  }

  # per-trace quota
  quota := {"emailApi": 2}
  deny["quota exceeded"] {
    input.counters[input.action] >= quota[input.action]
  }

```

E Policy mis-specification, coverage, and OPA-in-loop cost

The DPA guarantee is *policy-relative*: the soundness of Thm. 1 holds with respect to the supplied policy (assumption A3, §5). Mis-specification shifts errors predictably and visibly. An over-permissive policy, such as a missing prohibition or an over-broad grant, admits the corresponding action, a false pass. An over-restrictive policy, such as an unnecessary precondition, blocks a benign action, a false block that surfaces as an explicit blocked or abstain state. The monitor never issues a silent downgrade. Because the monitor reports the failed gate, a mis-specified policy stays auditable, and the abstain valve routes contested cases to human confirmation. Coverage is therefore an authoring concern. Nameable properties (permissions, prohibitions, explicit and laundered flow at field-tag granularity, quotas, and end-of-trace obligations) are enforced deterministically, and the conformal guard bounds the remaining residual under exchangeability, abstaining on actions that score above τ .

On overhead, the microsecond-scale cost reported in the main paper is the released in-process automaton, which evaluates the gates inside the controller. An OPA-backed deployment would add the cost of constructing the JSON state document and one OPA evaluation per decision. The guard runs on the proposer’s plan before execution, once per plan. This cost stays off the request data path. We do not benchmark the OPA round-trip and make no microsecond claim for it.

F Positioning across runtime-governance classes

DEON occupies a specific niche among runtime-governance approaches and complements the main lines. SMT-grounded runtime verification [19,4] compiles rich temporal and value-flow policies and blocks at invocation with precise witnesses. It is more expressive for cross-call value constraints and complex temporal patterns, at higher cost, and it adds no calibrated residual-risk layer. A richer symbolic monitor could in fact replace the DPA beneath the same conformal guard and ABSTAIN. Lattice and provenance information-flow enforcement [8] supports recursive provenance and cross-agent flow over dependency graphs, where DEON’s field-tag taint is lighter and coarser. The two complement each other. Ontology and deontic policy engines [35,21] manage permission and obligation lifecycles with semantic reasoning, whereas DEON uses a streaming au-

tomaton and adds distribution-free selective admission. Cryptographically authenticated workflows [10] strengthen boundary integrity, and DEON complements them, since per-action semantic checks and calibrated abstention remain useful even with an authenticated context. Rule-based tool-boundary guards [37] block at the tool call by induced rules, which overlaps the DPA’s pre-invocation blocking. Those guards leave the residual bound, calibrated abstain, and obligation lifecycle to DEON. Across these lines, DEON’s distinctive combination is formal per-action enforcement for policy-expressible violations together with a distribution-free, exchangeability-conditional guarantee on the residual, plus a first-class abstain and an auditable blocked state.